United States Patent	12411774
Kind Code	B2
Date of Patent	September 09, 2025
Inventor(s)	Mola; Jordi et al.

Treating main memory as a collection of tagged cache lines for trace logging

Abstract

Treating main memory as a collection of tagged cache lines for trace logging. A computer system allocates a plurality of memory blocks, and a corresponding plurality of tags, within a main memory. Each tag indicates whether data stored in a corresponding memory block has been captured by an execution trace. The computer system synchronizes these tags with tags in a memory cache and manages a traced status of the memory blocks. This can include one or more of (i) setting a tag to indicate a memory block has not been captured based on identifying a direct memory access operation, (ii) setting a tag based on whether a paged-in value of a memory block has been captured, (iii) setting a tag or memory categorization based whether a memory block has been initialized, or (iv) setting a tag or memory categorization based whether a memory block is mapped to a file.

Inventors: **Mola; Jordi (Bellevue, WA), Speier; Thomas Philip (Wake Forest, NC)**

Applicant: **Microsoft Technology Licensing, LLC** (Redmond, WA)

Family ID: **82899641**

Assignee: **Microsoft Technology Licensing, LLC (Redmond, WA)**

Appl. No.: **17/324776**

Filed: **May 19, 2021**

Prior Publication Data

Document Identifier	Publication Date
US 20220269614 A1	Aug. 25, 2022

Related U.S. Application Data

continuation-in-part parent-doc US 17229645 20210413 US 11561896 child-doc US 17324776
continuation-in-part parent-doc US 17229718 20210413 ABANDONED child-doc US 17324776
us-provisional-application US 63152240 20210222

Publication Classification

Int. Cl.: **G06F12/00** (20060101); **G06F12/0802** (20160101); **G06F12/0895** (20160101); **G06F12/12** (20160101); **G06F13/28** (20060101)

U.S. Cl.:

CPC **G06F12/0895** (20130101); **G06F12/12** (20130101); **G06F13/28** (20130101); G06F2212/6012 (20130101)

Field of Classification Search

CPC: G06F (12/0895); G06F (12/12); G06F (13/28); G06F (2212/6012)

USPC: 711/133

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
5249231	12/1992	Covey	713/193	G06F 12/1466
5269017	12/1992	Hayden	714/15	G06F 11/1407
6088773	12/1999	Kano	711/146	G06F 11/1407
6549959	12/2002	Yates	711/146	G06F 9/45558
2006/0179346	12/2005	Bishop	714/13	G06F 11/1407
2006/0212657	12/2005	Tuel	711/E12.012	G06F 12/0276
2011/0047334	12/2010	Eichenberger	711/E12.001	G06F 11/1405
2012/0246456	12/2011	Aloni	713/2	G06F 9/4401
2014/0237454	12/2013	Delporte	717/127	G06F 11/366
2016/0092225	12/2015	Collura	712/228	G06F 9/3851
2018/0060215	12/2017	Mola	N/A	G06F 11/3636
2019/0087305	12/2018	Mola	N/A	G06F 11/3636
2019/0258556	12/2018	Mola	N/A	N/A
2019/0266086	12/2018	Mola	N/A	G06F 11/3476
2019/0266090	12/2018	Mola	N/A	N/A

2020/0026639	12/2019	Mola	N/A	G06F 11/3636
2022/0269604	12/2021	Mola	N/A	G06F 12/0888
2022/0269615	12/2021	Mola et al.	N/A	N/A
2023/0038186	12/2022	Mola et al.	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
2019164710	12/2018	WO	N/A
2019164730	12/2018	WO	N/A

OTHER PUBLICATIONS

Satish Narayanasamy et al: “BugNet”, 2013 21st International Conference on Program Comprehension (ICPC); [International Symposium on Computer Architecture (ISCA)], IEEE, US May 1, 2005 (May 1, 2005), pp. 284-295, XP058168688 ISSN: 1063-6897, DOI: 10.1109/ISCA.2005.16 ; ISBN: 978-0-7695-3174-8 (Year: 2005). cited by examiner

Dongyoon Lee et al: “Offline symbolic analysis for multi-processor execution replay”, Microarchitecture, 2009. MICRO-42. 42nd Annual IEEE/ACM International Symposium on, IEEE, Piscataway, NJ USA, Dec. 12 (Dec. 12, 2009), pp. 564-575, XP031597154, ISBN: 978-1-60558-798-1 (Year: 2009). cited by examiner

Min Xu et al: “A “flight data recorder” for enabling full-system multiprocessor deterministic replay”, Proceedings of the 30th. International Symposium on Computer Architecture. ISCA 2003. San Diego, CA, May 1, 2003, pp. 122-135, XP058251437; ISBN: 978-0-7695-1945-6 (Year: 2003). cited by examiner

Lee, et al., “Offline Symbolic Analysis for Multi-Processor Execution Replay”, In Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture, Dec. 12, 2009, pp. 564-575. cited by applicant

Narayanasamy, et al., “BugNet: Continuously Recording Program Execution for Deterministic Replay Debugging”, In Proceedings of the 32nd International Symposium on Computer Architecture, Jun. 4, 2005, 12 Pages. cited by applicant

“International Search Report and Written Opinion Issued in PCT Application No. PCT/US2022/013441”, Mailed Date: Mar. 25, 2022, 12 Pages. (MS# 409626-WO-PCT). cited by applicant

“International Search Report and Written Opinion Issued in PCT Application No. PCT/US2022/013443”, Mailed Date: Mar. 25, 2022, 12 Pages. (MS# 409391-WO-PCT). cited by applicant

“International Search Report and Written Opinion Issued in PCT Application No. PCT/US2022/013444”, Mailed Date: Mar. 25, 2022, 12 Pages. (MS# 409626A-WO-PCT). cited by applicant

Xu, et al., “A “Flight Data Recorder” for Enabling Full-System Multiprocessor Deterministic Replay”, In Proceedings of the 30th Annual International Symposium on Computer Architecture, Jun. 9, 2003, 12 Pages. cited by applicant

“Notice of Allowance Issued in U.S. Appl. No. 17/229,645”, Mailed Date : Sep. 27, 2022, 8 Pages. (MS# 409391-US-NP). cited by applicant

“Final Office Action Issued in U.S. Appl. No. 17/229,718”, Mailed Date: Oct. 21, 2022, 17 Pages. (MS# 409626-US-NP). cited by applicant

“Non Final Office Action Issued in U.S. Appl. No. 17/229,645”, Mailed Date: Jun. 15, 2022, 11 Pages. (MS# 409391-US-NP). cited by applicant

“Non Final Office Action Issued in U.S. Appl. No. 17/229,718”, Mailed Date: Feb. 3, 2023, 16

Pages. (MS# 409626-US-NP). cited by applicant
"Notice of Allowance Issued in U.S. Appl. No. 17/975,410", Mailed Date: Feb. 22, 2023, 13 Pages. (MS# 409391-US-ONT). cited by applicant
"Non Final Office Action Issued in U.S. Appl. No. 17/229,718", Mailed Date: Jul. 11, 2022, 16 Pages. (MS# 409626-US-NP). cited by applicant
U.S. Appl. No. 17/229,645, filed Apr. 13, 2021. cited by applicant
U.S. Appl. No. 63/152,240, filed Feb. 22, 2021. cited by applicant
U.S. Appl. No. 17/229,718, filed Apr. 13, 2021. cited by applicant
U.S. Appl. No. 17/975,410, filed Oct. 27, 2022. cited by applicant
"Final Office Action Issued in U.S. Appl. No. 17/229,718", Mailed Date: Jun. 23, 2023, 19 Pages. (MS# 409626-US-NP). cited by applicant
Communications under 71(3) received for European Application No. 22704117.5, mailed on Jun. 12, 2024, 8 pages. cited by applicant
Decision to grant a European patent pursuant to Article 97(1) received in European Application No. 22704117.5, mailed on Oct. 24, 2024, 2 pages. cited by applicant
Bhansali, et al., "Framework for Instruction-level Tracing and Analysis of Progam Executions", In Proceedings of the 2nd International Conference on Virtual Execution Environments, Jun. 14, 2006, pp. 154-163. cited by applicant
Communication pursuant to Article 94(3) Received in European Patent Application No. 22703785.0, mailed on Mar. 12, 2025, 06 pages. cited by applicant
Narayansamy, et al., "Automatic logging of operating system effects to guide application-level architecture simulation", Proceedings of the joint international conference on Measurement and modeling of computer systems, Jun. 26, 2006, pp. 216-227. cited by applicant

---

Primary Examiner: Wong; Titus

Attorney, Agent or Firm: Workman Nydegger

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS (1) This application is a continuation-in-part of U.S. patent application Ser. No. 17/229,718, filed Apr. 13, 2021, entitled "CACHE-BASED TRACE LOGGING USING TAGS IN SYSTEM MEMORY," and of U.S. patent application Ser. No. 17/229,645, filed Apr. 13, 2021, entitled "CACHE-BASED TRACE LOGGING USING TAGS IN AN UPPER-LEVEL CACHE," each of which claims priority to, and the benefit of, U.S. Provisional Patent Application Ser. No. 63/152,240, filed Feb. 22, 2021, entitled "CACHE-BASED TRACE LOGGING USING TAGS IN A HIGHER MEMORY TIER." The entire contents of each of these applications are incorporated by reference herein in their entireties.

BACKGROUND
(1) Tracking down and correcting undesired software behaviors is a core activity in software development. Undesired software behaviors can include many things, such as execution crashes, runtime exceptions, slow execution performance, incorrect data results, data corruption, and the like. Undesired software behaviors are triggered by a vast variety of factors such as data inputs, user inputs, race conditions (e.g., when accessing shared resources), etc. Given the variety of triggers, undesired software behaviors are often rare and seemingly random, and extremely difficult to reproduce. As such, it is often very time-consuming and difficult for a developer to identify a given undesired software behavior. Once an undesired software behavior has been identified, it is again often time-consuming and difficult to determine its root cause (or causes).

(2) Developers use a variety of approaches to identify undesired software behaviors, and to then identify one or more locations in an application's code that cause the undesired software behavior. For example, developers often test different portions of an application's code against different inputs (e.g., unit testing). As another example, developers often reason about execution of an application's code in a debugger (e.g., by setting breakpoints/watchpoints, by stepping through lines of code, etc. as the code executes). As another example, developers often observe code execution behaviors (e.g., timing, coverage) in a profiler. As another example, developers often insert diagnostic code (e.g., trace statements) into the application's code.
(3) While conventional diagnostic tools (e.g., debuggers, profilers, etc.) have operated on “live” forward-executing code, an emerging form of diagnostic tools enable “historic” debugging (also referred to as “time travel” or “reverse” debugging), in which the execution of at least a portion of an execution context is recorded into one or more trace files (i.e., an execution trace). Using some tracing techniques, an execution trace can contain “bit-accurate” historic execution trace data, which enables any recorded portion the traced execution context to be virtually “replayed” (e.g., via emulation) down to the granularity of individual instructions (e.g., machine code instructions, intermediate language code instructions, etc.). Thus, using “bit-accurate” trace data, diagnostic tools enable developers to reason about a recorded prior execution of subject context, as opposed to conventional debugging which is limited to a “live” forward execution. For example, using replayable execution traces, some historic debuggers provide user experiences that enable both forward and reverse breakpoints/watch points, that enable code to be stepped through both forwards and backwards, etc. Some historic profilers, on the other hand, are able to derive code execution behaviors (e.g., timing, coverage) from prior-executed code.
(4) Some techniques for recording execution traces operate based largely on recording influxes to a microprocessor's (processor's) memory cache. However, since modern processors commonly execute at the rate of tens-to hundreds-of thousands of MIPS (millions of instructions per second), replayable execution traces of a program's thread can capture vast amounts of information, even if mere fractions of a second of the thread's execution are captured. As such, replayable execution traces quickly grow very large in size in memory and/or on disk.

BRIEF SUMMARY

(5) At least some embodiments described herein reduce the size of replayable execution traces by performing cache-based trace logging using tags in a higher memory tier. In general, these embodiments operate to log influxes to a recording first cache level, but leverage tags within a higher memory tier to track whether a value of a given cache line influx is already captured by an execution trace (or is otherwise recoverable). In some embodiments, during an influx of a cache line to the first cache level, tracing logic consults a tag in the higher memory tier to determine if the value that is being influxed can be reconstructed from prior trace logging, such as trace logging performed in connection with a prior influx of the cache line to the first cache level. If so, these embodiments refrain from capturing a current value of the cache line into the execution trace when influxing the cache line to the first cache level. Additionally, during evictions from the first cache level, these embodiments determine whether the cache line being evicted is in a “logged state” within the first cache level (i.e., a current value of the cache line can be obtained from a prior-recorded trace, and/or can be constructed by replaying the prior-recorded trace) and sets a tag in the higher memory tier as appropriate to indicate whether or not the cache line that is being evicted is logged. In embodiments, performing cache-based trace logging to track whether a value of a given cache line influx is already captured by an execution trace has a technical effect of reducing a number of cache influxes that are recorded into an execution trace, which in turn has additional technical effects of reducing a size of the execution trace as compared to prior tracing techniques, and of reducing processor utilization for carrying out the recording of cache influxes as compared to prior tracing techniques.
(6) In some embodiments, the tags within the higher memory tier track whether a value of a cache

line that is being influxed from an upper-level cache is already captured by an execution trace. In embodiments, one or more of the tags within the higher memory tier are stored in an upper second cache level, and each of those tags indicate whether a value of a corresponding cache line in the second cache level is already captured by an execution trace. Additionally, or alternatively, in embodiments, one or more of the tags within the higher memory tier are stored in main memory, and each of those tags indicate whether a value of a corresponding cache line in the second cache level is already captured by an execution trace. Experimentally, using tags (whether stored in the second cache level or in main memory) to indicate whether a value of a corresponding cache line in the second cache level is already captured by an execution trace have been shown to reduce trace file size by 3-4×, when compared to prior tracing techniques that lack use of tags in a higher memory tier.

(7) Additionally, or alternatively, in some embodiments one or more of the tags within the higher memory tier are stored in main memory, and each of those tag(s) indicate whether a value of a block of memory in the main memory is already captured by an execution trace (or is otherwise recoverable). Thus, these embodiments operatively treat main memory as a collection of tagged cache lines for trace logging. In these embodiments, a computer system allocates a plurality of memory blocks within the main memory, along with a corresponding plurality of tags—each of which indicates whether data stored in its associated memory block has been captured by an execution trace (or is otherwise recoverable). The computer system synchronizes these tags with a memory cache (such that the foregoing tagging techniques remain operable) and manages those tags in light of memory operations affecting the memory blocks—such as direct memory access (DMA) operations, memory paging operations, memory initialization operations, and/or file-mapping operations. In embodiments, treating main memory as a collection of tagged cache lines for trace logging has a further technical effect of using memory to increase a number of values that can be tracked as being previously captured, or not. This, in turn, further reduces the number of cache influxes that are recorded into an execution trace (further reducing a size of the execution trace, and further reducing processor utilization for carrying out the recording of cache influxes). Experimentally, treating main memory as a collection of tagged cache lines for trace logging has been shown to reduce trace file size by 10-100×, when compared to using tags to indicate whether a value of a corresponding cache line in the second cache level is already captured by an execution trace.

(8) In accordance with the foregoing, some embodiments are directed to methods, systems, and computer program products for treating main memory as a collection of tagged cache lines for trace logging. In these embodiments, a computer system allocates a plurality of memory blocks within a main memory, including allocating an amount of memory for each of the plurality of memory blocks that is a size of each of a plurality of cache lines in a memory cache. The computer system allocates also allocates a plurality of tags within the main memory. Each tag is associated with one of the plurality of memory blocks and indicates whether data stored in its associated memory block has been captured by an execution trace. The computer system also synchronizes at least one of the plurality of tags within the main memory with at least one cache line tag within the memory cache and manages a traced status of at least one memory block. In some embodiments, managing the traced status of at least one memory block comprises, based at least on identifying a DMA operation modifying a particular memory block of the plurality of memory blocks, setting a particular tag of the plurality of tags that is associated with the particular memory block to a value indicating that data stored in the particular memory block has not been captured by the execution trace. In some embodiments, managing the traced status of at least one memory block comprises, based at least on identifying a memory page-in operation affecting the particular memory block, setting the particular tag based on whether a paged-in value has been captured by the execution trace. In some embodiments, managing the traced status of at least one memory block comprises, based at least on initializing the data stored in the particular memory block, performing one or

more of (i) setting an indication that a memory page containing the particular memory block is not logged, or (ii) setting the particular tag to a value indicating that data stored in the particular memory block has been captured by the execution trace. In some embodiments, managing the traced status of at least one memory block comprises, based at least on associating the particular memory block with a file, performing one or more of (i) setting an indication that the memory page containing the particular memory block is not logged, or (ii) setting the particular tag to a value indicating that data stored in the particular memory block has been captured by the execution trace.
(9) This summary is provided to introduce a selection of concepts in a simplified form that are further described below in the Detailed Description. This Summary is not intended to identify key features or essential features of the claimed subject matter, nor is it intended to be used as an aid in determining the scope of the claimed subject matter.

Description

BRIEF DESCRIPTION OF THE DRAWINGS
(1) In order to describe the manner in which the above-recited and other advantages and features of the invention can be obtained, a more particular description of the invention briefly described above will be rendered by reference to specific embodiments thereof which are illustrated in the appended drawings. Understanding that these drawings depict only typical embodiments of the invention and are not therefore to be considered to be limiting of its scope, the invention will be described and explained with additional specificity and detail through the use of the accompanying drawings in which:
(2) FIG. **1**A illustrates an example computing environment that facilitates cache-based trace logging using tags in a higher memory tier;
(3) FIG. **1**B illustrates additional detail of control logic for cache-based trace logging using tags in a higher memory tier;
(4) FIG. **1**C illustrates additional detail of a memory tagging component that treats main memory as a collection of tagged cache lines while trace logging using tags in a higher memory tier;
(5) FIG. **2** illustrates an example environment demonstrating multi-level caches;
(6) FIG. **3** illustrates an example of a processor cache that includes a plurality of entries;
(7) FIG. **4** illustrates an example of an execution trace;
(8) FIG. **5**A illustrates a flow chart of an example method for using tags in a higher memory tier to determine whether or not to log a cache line influx;
(9) FIG. **5**B illustrates a flow chart of an example method for setting tags in a higher memory tier during a cache line eviction; and
(10) FIG. **6** illustrates a flow chart of an example method for treating main memory as a collection of tagged cache lines for trace logging.
DETAILED DESCRIPTION
(11) At least some embodiments described herein reduce the size of replayable execution traces by performing cache-based trace logging using tags in a higher memory tier. In general, these embodiments operate to log influxes to a recording first cache level, but leverage tags within a higher memory tier to track whether a value of a given cache line influx is already captured by an execution trace (or is otherwise recoverable). In some embodiments, during an influx of a cache line to the first cache level, tracing logic consults a tag in the higher memory tier to determine if the value that is being influxed can be reconstructed from prior trace logging, such as trace logging performed in connection with a prior influx of the cache line to the first cache level. If so, these embodiments refrain from capturing a current value of the cache line into the execution trace when influxing the cache line to the first cache level. Additionally, during evictions from the first cache level, these embodiments determine whether the cache line being evicted is in a “logged state”

within the first cache level (i.e., a current value of the cache line can be obtained from a prior-recorded trace, and/or can be constructed by replaying the prior-recorded trace) and sets a tag in the higher memory tier as appropriate to indicate whether or not the cache line that is being evicted is logged.

(12) In some embodiments, the tags within the higher memory tier track whether a value of a cache line that is being influxed from an upper-level cache is already captured by an execution trace. In embodiments, one or more of the tags within the higher memory tier are stored in an upper second cache level, and each of those tags indicate whether a value of a corresponding cache line in the second cache level is already captured by an execution trace. Additionally, or alternatively, in embodiments, one or more of the tags within the higher memory tier are stored in main memory, and each of those tags indicate whether a value of a corresponding cache line in the second cache level is already captured by an execution trace.

(13) Additionally, or alternatively, in some embodiments one or more of the tags within the higher memory tier are stored in main memory, and each of those tag(s) indicate whether a value of a block of memory in the main memory is already captured by an execution trace (or is otherwise recoverable). Thus, these embodiments operatively treat main memory as a collection of tagged cache lines for trace logging.

(14) To the accomplishment of these (and other) embodiments, FIG. **1**A illustrates an example computing environment **100***a* that facilitates cache-based trace logging using tags in a higher memory tier. In particular, computing environment **100***a* includes a special-purpose or general-purpose computer system **101**, which includes at least one processor **102** that is configured to perform a hardware-based execution trace logging, based on recording influxes to at least one level of a cache. As shown, in addition to processor(s) **102**, computer system **101** also includes at least main memory **103** (often referred to as system memory or primary memory) and durable storage **104**, which are communicatively coupled to each other, and to the processor(s) **102**, using at least one communications bus **105**.

(15) Embodiments within the scope of the present invention can include physical and other computer-readable media for carrying or having stored thereon computer-executable instructions and/or data structures. Such computer-readable media can be any available media that can be accessed by a general-purpose or special-purpose computer system. Computer-readable media that store computer-executable instructions and/or data structures are computer storage media. Computer-readable media that carry computer-executable instructions and/or data structures are transmission media. Thus, by way of example, and not limitation, embodiments of the invention can comprise at least two distinctly different kinds of computer-readable media: computer storage media and transmission media.

(16) Computer storage media are physical storage media (e.g., main memory **103** and/or durable storage **104**) that store computer-executable instructions and/or data structures. Physical storage media include computer hardware, such as RAM, ROM, EEPROM, solid state drives (“SSDs”), flash memory, phase-change memory (“PCM”), optical disk storage, magnetic disk storage or other magnetic storage devices, or any other hardware storage device(s) which can be used to store program code in the form of computer-executable instructions or data structures, which can be accessed and executed by a general-purpose or special-purpose computer system to implement the disclosed functionality of the invention.

(17) Transmission media can include a network and/or data links which can be used to carry program code in the form of computer-executable instructions or data structures, and which can be accessed by a general-purpose or special-purpose computer system. A “network” is defined as one or more data links that enable the transport of electronic data between computer systems and/or modules and/or other electronic devices. When information is transferred or provided over a network or another communications connection (either hardwired, wireless, or a combination of hardwired or wireless) to a computer system, the computer system may view the connection as

transmission media. Combinations of the above should also be included within the scope of computer-readable media.

(18) Further, upon reaching various computer system components, program code in the form of computer-executable instructions or data structures can be transferred automatically from transmission media to computer storage media (or vice versa). For example, computer-executable instructions or data structures received over a network or data link can be buffered in RAM within a network interface module (not shown), and then eventually transferred to computer system RAM (e.g., main memory **103**) and/or to less volatile computer storage media (e.g., durable storage **104**) at the computer system. Thus, it should be understood that computer storage media can be included in computer system components that also (or even primarily) utilize transmission media.

(19) Computer-executable instructions comprise, for example, instructions and data which, when executed at one or more processors, cause a general-purpose computer system, special-purpose computer system, or special-purpose processing device to perform a certain function or group of functions. Computer-executable instructions may be, for example, machine code instructions (e.g., binaries), intermediate format instructions such as assembly language, or even source code.

(20) Those skilled in the art will appreciate that the invention may be practiced in network computing environments with many types of computer system configurations, including, personal computers, desktop computers, laptop computers, message processors, hand-held devices, multi-processor systems, microprocessor-based or programmable consumer electronics, network PCs, minicomputers, mainframe computers, mobile telephones, PDAs, tablets, pagers, routers, switches, and the like. The invention may also be practiced in distributed system environments where local and remote computer systems, which are linked (either by hardwired data links, wireless data links, or by a combination of hardwired and wireless data links) through a network, both perform tasks. As such, in a distributed system environment, a computer system may include a plurality of constituent computer systems. In a distributed system environment, program modules may be located in both local and remote memory storage devices.

(21) Those skilled in the art will also appreciate that the invention may be practiced in a cloud computing environment. Cloud computing environments may be distributed, although this is not required. When distributed, cloud computing environments may be distributed internationally within an organization and/or have components possessed across multiple organizations. In this description and the following claims, "cloud computing" is defined as a model for enabling on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services). The definition of "cloud computing" is not limited to any of the other numerous advantages that can be obtained from such a model when properly deployed.

(22) A cloud computing model can be composed of various characteristics, such as on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service, and so forth. A cloud computing model may also come in the form of various service models such as, for example, Software as a Service ("SaaS"), Platform as a Service ("PaaS"), and Infrastructure as a Service ("IaaS"). The cloud computing model may also be deployed using different deployment models such as private cloud, community cloud, public cloud, hybrid cloud, and so forth.

(23) Some embodiments, such as a cloud computing environment, may comprise a system that includes one or more hosts that are each capable of running one or more virtual machines. During operation, virtual machines emulate an operational computing system, supporting an operating system and perhaps one or more other applications as well. In some embodiments, each host includes a hypervisor that emulates virtual resources for the virtual machines using physical resources that are abstracted from view of the virtual machines. The hypervisor also provides proper isolation between the virtual machines. Thus, from the perspective of any given virtual machine, the hypervisor provides the illusion that the virtual machine is interfacing with a physical resource, even though the virtual machine only interfaces with the appearance (e.g., a virtual

resource) of a physical resource. Examples of physical resources including processing capacity, memory, disk space, network bandwidth, media drives, and so forth.

(24) As shown in FIG. **1**A, in embodiments each processor **102** includes at least one processing unit **106**, at least one cache **107** (memory cache), and control logic **108** (e.g., gate logic, executable microcode, etc.). Each processing unit **106** (e.g., processor core) loads and executes machine code instructions at one or more execution units **106***b*. During execution of these machine code instructions, the instructions can use internal processor registers **106***a* as temporary storage locations, and can read and write to various locations in main memory **103** via the cache **107**. Each processing unit **106** in a given processor **102** executes machine code instructions that are defined by a processor instruction set architecture (ISA). The particular ISA of each processor **102** can vary based on processor manufacturer and processor model. Common ISAs include the IA-64 and IA-32 architectures from INTEL, INC., the AMD64 architecture from ADVANCED MICRO DEVICES, INC., and various Advanced RISC Machine ("ARM") architectures from ARM HOLDINGS, PLC, although a great number of other ISAs exist and can be used by the present invention. In general, a machine code instruction is the smallest externally-visible (i.e., external to the processor) unit of code that is executable by a processor.

(25) Registers **106***a* are hardware storage locations that are defined based on the ISA of the processor **102**. In general, registers **106***a* are read from and/or written to by machine code instructions, or a processing unit **106**, as those instructions execute at execution units **106***b*. Registers **106***a* are commonly used to store values fetched from the cache **107** for use as inputs to executing machine code instructions, to store the results of executing machine code instructions, to store a program instruction count, to support maintenance of a thread stack, etc. In some embodiments, registers **106***a* include "flags" that are used to signal some state change caused by executing machine code instructions (e.g., to indicate if an arithmetic operation cased a carry, a zero result, etc.). In some embodiments, registers **106***a* include one or more control registers (e.g., which are used to control different aspects of processor operation), and/or other processor model-specific registers (MSRs).

(26) The cache **107** temporarily caches blocks of main memory **103** during execution of machine code instructions by one or more of processing units **106**. In embodiments, the cache **107** includes one or more "code" portions that cache portions of main memory **103** storing application code, as well as one or more "data" portions that cache portions of main memory **103** storing application runtime data. If a processing unit **106** requests data (e.g., code or application runtime data) not already stored in the cache **107**, then the processing unit **106** initiates a "cache miss," causing block(s) of data to be fetched from main memory **103** and influxed into the cache **107**—while potentially replacing and "evicting" some other data already stored in the cache **107** back to main memory **103**.

(27) In the embodiments herein, the cache **107** comprises multiple cache levels (sometimes referred to cache tiers or cache layers)—such as a level 1 (L1) cache, a level 2 (L2) cache, a level 3 (L3) cache, etc. For example, FIG. **2** illustrates an example computing environment **200** demonstrating multi-level caches. In FIG. **2**, the example computing environment **200** comprises two processors **201**—processor **201***a* and processor **201***b* (e.g., processor(s) **102** of FIG. **1**A) and a main memory **202** (e.g., main memory **103** of FIG. **1**A). In the example computing environment **200**, each processor **201** comprises four processing units (e.g., processing unit(s) **106** of FIG. **1**A), including processing units A1-A4 for processor **201***a* and processing units B1-B4 for processor **201***b*.

(28) In example computing environment **200**, each processor **201** also includes a three-level cache hierarchy. Computing environment **200** illustrates one example cache layout only, and it is not limiting to the cache hierarchies in which the embodiments herein may operate. In computing environment **200**, each processing unit includes its own dedicated L1 cache (e.g., L1 cache "L1-A1" in processor **201***a* for unit A1, L1 cache "L1-A2" in processor **201***a* for unit A2, etc.). Relative to the L1 caches, each processor **201** also includes two upper-level L2 caches (e.g., L2 cache "L2-

A1" in processor **201**_a_ that serves as a backing store for L1 caches L1-A1 and L1-A2, L2 cache "L1-A2" in processor **201**_a_ that serves as a backing store for L1 caches L1-A3 and L1-A4, etc.). Finally, relative to the L2 caches, each processor **201** also includes a single L3 cache (e.g., L3 cache "L3-A" in processor **201**_a_ that serves as a backing store for L2 caches L2-A1 and L2-A2, and L3 cache "L3-B" in processor **201**_b_ that serves as a backing store for L2 caches L2-B1 and L2-B2).

(29) As shown, main memory **202** serves as a backing store for the L3 caches L3-A and L3-B. In this arrangement, and depending on cache implementation, cache misses in an L1 cache might be served by its corresponding L2 cache, its corresponding L3 cache, and/or main memory **202**; cache misses in an L2 cache might be served by its corresponding L3 cache and/or main memory **202**; and cache misses in an L3 cache might be served by main memory **202**.

(30) In some environments, some cache levels exist separate from a processor; for instance, in computing environment **200** one or both of the L3 caches could alternatively exist separate from processors **201**, and/or computing environment **200** could include one or more additional caches (e.g., L4, L5, etc.) that exist separate from processors **201**.

(31) As demonstrated by the arrows within each processor **201**, when multiple cache levels exist, each processing unit typically interacts directly with the lowest level (e.g., L1). In many implementations, data flows between the levels (e.g., an L3 cache interacts with the main memory **202** and serves data to an L2 cache, and the L2 cache in turn serves data to the L1 cache). However, as will be appreciated by one of ordinary skill in the art, the particular manner in which processing units interact with a cache, and the particular manner in which data flows between cache levels, may vary (e.g., depending on whether the cache is inclusive, exclusive, or some hybrid).

(32) Given their arrangement, the caches in computing environment **200** may be viewed as "shared" caches. For example, each L2 and L3 cache serves multiple processing units within a given processor **201** and are thus shared by these processing units. The L1 caches within a given processor **201**, collectively, can also be considered shared—even though each one corresponds to a single processing unit—because the individual L1 caches may coordinate with each other via a cache coherency protocol (CCP) to ensure consistency (i.e., so that each cached memory location is viewed consistently across all the L1 caches). The L2 caches within each processor **201** similarly may coordinate via a CCP. Additionally, each individual L1 cache may be shared by two or more physical or logical processing units, such as where the processor **201** supports hyper-threading, and are thus "shared" even at an individual level.

(33) In embodiments, each level of cache(s) **107** comprises a plurality of entries that store cache lines (also commonly referred to as cache blocks). Each cache line/block corresponds to a contiguous block of main memory **103**. For example, FIG. **3** illustrates an example **300** of a memory cache **301** (e.g., an L1 cache, an L2 cache, etc.) that includes a plurality of entries **303**. In example **300**, each entry **303** comprises at least an address portion **302**_a_ that stores a memory address and a cache line portion **302**_b_ that stores a block of data corresponding to that memory address. During an influx into a given entry **303** of the memory cache **301**, the entry's cache line portion **302**_b_ is generally filled with a block of data obtained from an upper-level cache, or from the main memory **103**. Depending on a size of the cache line portion **302**_b_, each entry **303** may potentially store data spanning a plurality of consecutive individually addressable locations within the main memory **103**. The cache line portion **302**_b_ of each entry **303** can be modified by one or more of processing units **106**, and eventually be evicted back to an upper-level cache, or to the main memory **103**. As indicated by the ellipses within the memory cache **301**, each cache can include a large number of entries. For example, a contemporary 64-bit INTEL processor may contain individual L1 caches for each processing unit **106** that each comprises 512, or more, entries. In such a cache, each entry is typically used to store a 64-byte (512-bit) value in reference to a 6-byte (48-bit) to 8-byte (64-bit) memory address. As shown in computing environment **200**, caches are generally larger in size (i.e., more cache entries) as their cache level increases. For

example, an L2 cache is generally larger than an L1 cache, an L3 cache is generally larger than an L2 cache, and so on.

(34) In some situations, the address portion **302***a* of each entry **303** stores a physical memory address, such as the actual corresponding physical memory address in the main memory **103**. In other situations, the address portion **302***a* of each entry **303** stores a virtual memory address. In embodiments, a virtual memory address is an address within a virtual address space that is exposed by an operating system to a process executing at the processor(s) **102**. This virtual address space provides one or more abstractions to the process, such as that the process has its own exclusive memory space and/or that the process has more memory available to it than actually exists within the main memory **103**. Such abstractions can be used, for example, to facilitate memory isolation between different processes executing at the processor(s) **102**, including isolation between user-mode processes and kernel-mode processes. In embodiments, virtual to physical memory address mappings are maintained within memory page tables that are stored in the main memory **103**, and that are managed by an operating system and/or hypervisor (e.g., operating environment **109**, described infra). In general, these memory page tables comprise a plurality of page table entries (PTEs) that map ranges (i.e., pages) of virtual memory addresses to ranges (i.e., pages) of physical memory addresses. In embodiments, each PTE stores additional attributes, or flags, about its corresponding memory pages, such as memory page permissions (e.g., read-only, writeable, etc.), page state (e.g., dirty, clean, etc.), and the like. In embodiments, one or more translation lookaside buffers (TLBs, not shown) within each processor **102** facilitates virtual addressing, and comprises a dedicated form of cache that stores recently obtained PTEs mapping virtual and physical memory pages, as obtained from the memory page tables stored in the main memory **103**. In some implementations, PTEs are part of a multi-level hierarchy, which includes one or more page directory entries (PDEs) that support discovery of individual PTEs. If a processor **102** lacks a TLB, then it may lack support for virtual memory addressing.

(35) As mentioned, caches coordinate using a CCP. In general, a CCP defines how consistency is maintained between various caches as various processing units read from and write data to those caches, and how to ensure that the processing units always read consistent data for a given cache line. CCPs are typically related to, and enable, a memory model defined by the processor's instruction set architecture (ISA). Examples of popular ISA's include the x86 and x86_64 families of architectures from INTEL, and the ARM architecture from ARM HOLDINGS. Examples of common CCPs include the MSI protocol (i.e., Modified, Shared, and Invalid), the MESI protocol (i.e., Modified, Exclusive, Shared, and Invalid), and the MOESI protocol (i.e., Modified, Owned, Exclusive, Shared, and Invalid). Each of these protocols define a state for individual cache line stored in a shared cache. A “modified” cache line contains data that has been modified in the shared cache and is therefore inconsistent with the corresponding data in the backing store (e.g., main memory **103** or another cache). When a cache line having the “modified” state is evicted from the shared cache, common CCPs require the cache to guarantee that its data is written back the backing store, or that another cache take over this responsibility. A “shared” cache line is not permitted to be modified, and may exist in a shared or owned state in another cache. The shared cache can evict this data without writing it to the backing store. An “invalid” cache line contains no valid data and can be considered empty and usable to store data from cache miss. An “exclusive” cache line contains data that matches the backing store and is used by only a single processing unit. It may be changed to the “shared” state at any time (i.e., in response to a read request) or may be changed to the “modified” state when writing to it. An “owned” cache location contains data that that is inconsistent with the corresponding data in the backing store. When a processing unit makes changes to an owned cache location, it notifies the other processing units—since the notified processing units may need to invalidate or update based on the CCP implementation.

(36) As shown, each entry in the memory cache **301** may include one or more additional portions **302***c*. In some embodiments, one additional portion **302***c* comprises one or more tracking bits used

to track whether a cache line stored in a corresponding entry **303** has been logged to a trace or not, as described infra. In some embodiments, an additional portion **302***c* stores a tag that comprises one or more data fields for storing information relevant to its corresponding entry **303**. In embodiments, the entries of at least one cache level comprises the additional portion **302***c* for storing tags, and those embodiments use those tags to improve trace logging, as described infra.
(37) Returning to FIG. **1**A, in embodiments, control logic **108** comprises microcode (i.e., executable instructions) and/or physical logic gates that control operation of the processor **102**. In general, control logic **108** functions as an interpreter between the hardware of the processor **102** and the processor ISA exposed by the processor **102** to executing applications (e.g., operating environment **109** and application(s) **113**) and controls internal operation of the processor **102**. In embodiments, the control logic **108** is embodied on on-processor storage, such as ROM, EEPROM, etc. In some embodiments, this on-processor storage is writable (in which case the control logic **108** is updatable), while in other embodiments this on-processor storage is read-only (in which case the control logic **108** cannot be updated).
(38) The durable storage **104** stores computer-executable instructions and/or data structures representing executable software components. Correspondingly, during execution of these software components at the processor(s) **102**, one or more portions of these computer-executable instructions and/or data structures are loaded into main memory **103**.
(39) For example, the durable storage **104** is illustrated as storing computer-executable instructions and/or data structures corresponding to an operating environment **109** and one or more application(s) **113**. Correspondingly, the main memory **103** is shown as storing one or more operating environment runtime(s) **109**′ (e.g., machine code instructions and/or runtime data supporting execution of the operating environment **109**), and as storing one or more application runtime(s) **113**′ (e.g., machine code instructions and/or runtime data supporting execution of one or more of application(s) **113**).
(40) The main memory **103** and durable storage **104** can also store other data, such as one or more replayable execution trace(s) (i.e., execution trace(s) **114**′ stored in main memory **103** and/or execution trace(s) **114** stored in durable storage **104**), and one or more of region categorizations **115**, memory blocks **116**, or tags **117**, each described infra. In embodiments, one or more of tags **117** indicates if the value of a corresponding memory block in memory blocks **116** is captured by an execution trace **114**. Additionally, or alternatively, one or more of tags **117** indicates if the value of a corresponding cache line in cache **107** is captured by an execution trace **114**.
(41) In embodiments, the operating environment **109** includes a hypervisor **109***a* and/or one or more operating system(s) **109***b*. Correspondingly, in embodiments, the operating environment runtime(s) **109**′ include a hypervisor runtime **109***a*′, and/or one or more operating system runtime(s) **109***b*′. For example, in some embodiments, the operating environment **109** comprises the hypervisor **109***a* executing directly on the hardware (e.g., processor(s) **102**, main memory **103**, and durable storage **104**) of computer system **101**, and one or more of the operating system(s) **109***b* executing on top of the hypervisor **109***a*. In other embodiments, however, the operating environment **109** comprises an operating system **109***b* executing directly on the hardware (e.g., processor(s) **102**, main memory **103**, and durable storage **104**) of computer system **101**.
(42) Additionally, in embodiments, the operating environment **109** includes a debugging component **110**. In various examples, the debugging component **110** is part of the hypervisor **109***a*, part of the operating system **109***b*, or a standalone component that is separate from both of the hypervisor **109***a* and the operating system **109***b*. As generally indicated by arrows between the debugging component **110** and the region categorizations **115**, the memory blocks **116**, and the tags **117**, in embodiments the debugging component **110** manages each of the region categorizations **115** (using the categorization component **111**), the memory blocks **116** (using memory tagging component **112**), and the tags **117** (using memory tagging component **112**).
(43) In embodiments, the debugging component **110** and the control logic **108** cooperate to record

one or more replayable execution trace(s) **114**/**114′** of code execution at the processor(s) **102**. In embodiments, tracing techniques utilized by the operating environment **109** and control logic **108** to record replayable execution traces **114**/**114′** are based at least on the processor(s) **102** recording influxes to at least a portion of their cache(s) **107** during code execution. In embodiments, each replayable execution trace **114**/**114′** comprises a “bit-accurate” record of execution of a corresponding context (e.g., process, operating system, virtual machine, enclave, hypervisor, etc.) as that context executed at the processor(s) **102**. As used herein, a replayable execution trace is a “bit accurate” record of that context's execution activity. This bit-accurate record enables machine code instructions that were previously executed as part of the context at the processing unit(s) **106** to be replayed later, such that, during replay, these machine code instructions are re-executed in the same order, and consume the same data that they did during trace recording. While a variety of bit-accurate tracing approaches are possible, as mentioned, the embodiments herein record a bit-accurate execution trace based on logging at least some of the influxes to cache(s) **107** during execution of a traced context (e.g., process, virtual machine, etc.). By logging at least some of these influxes during execution of the context, a replayable execution trace **114**/**114′** of that context captures at least some of the memory reads that were performed by the machine code instructions that executed as part of the context.

(44) The cache-based tracing techniques used by the embodiments herein are built upon an observation that each processor **102** (including its the cache(s) **107**) form a semi- or quasi-closed system. For example, once portions of data for an executing context (i.e., machine code instructions and runtime data) are loaded into a processor's cache(s) **107**, a processing unit **106** can continue executing that context—without any other external input—as a semi- or quasi-closed system for bursts of time. In particular, once the cache(s) **107** are loaded with machine code instructions and runtime data, the execution unit **106***b* can load and execute those machine code instructions from the cache(s) **107**, using runtime data stored in the cache(s) **107** as input to those machine code instructions, and using the registers **106***a*. So long as the data (i.e., machine code instructions and runtime data) that are needed for the processor **102** to execute that thread exists within the cache(s) **107**, the processor **102** can continue executing that context without further external input.

(45) When a processing unit **106** needs some influx of data (e.g., because a machine code instruction it is executing, will execute, or may execute accesses code or runtime data not already in the cache(s) **107**), the processor **102** may execute a “cache miss,” importing data into the cache(s) **107** from the main memory **103**. For example, if a data cache miss occurs when a processing unit **106** executes a machine code instruction that performs a memory operation on a memory address within application runtime **113′** storing runtime data, the processor **102** imports runtime data from that memory address in the main memory **103** to one of the cache lines of the data portion of the cache(s) **107**. Similarly, if a code cache miss occurs when a processing unit **106** tries to fetch a machine code instruction from a memory address within application runtime **113′** storing application code, the processor **102** imports code data from that memory address in main memory **103** to one of the cache lines of the code portion of the cache(s) **107**. The processing unit **106** then continues execution using the newly imported data, until new data is needed.

(46) In embodiments, each processor **102** is enabled to record a bit-accurate representation of execution of a context executing at the processor **102**, by recording, into a trace data stream corresponding to the context, sufficient data to be able to reproduce the influxes of information into the processor's cache(s) **107** as the processor's processing units **106** execute that context's code. For example, some approaches to recording these influxes operate on a per-processing-unit basis. These approaches involve recording, for each processing unit that is being traced, at least a subset of cache misses within the cache(s) **107**, along with a time during execution at which each piece of data was brought into the cache(s) **107** (e.g., using a count of instructions executed or some other counter). In some embodiments, these approaches involve also recording, for each processing unit that is being traced, any un-cached reads (i.e., reads from hardware components and un-cacheable

memory that bypass the cache(s) **107**) caused by that processing unit's activity, as well as the side-effects of having executed any non-deterministic processor instructions (e.g., one or more values of register(s) **106***a* after having executed a non-deterministic processor instruction).

(47) FIG. **4** illustrates an example of an execution trace (e.g., one of execution trace(s) **114**/**114′**). In particular, FIG. **4** illustrates an execution trace **400** that includes a plurality of data streams **401** (i.e., data streams **401***a*-**401***n*). In embodiments, each data stream **401** represents execution of a different context, such as a different thread that executed from the code of one of application(s) **113**. In an example, data stream **401***a* records execution of a first thread of an application **113**, while data stream **401***n* records an nth thread of the application **113**. As shown, data stream **401***a* comprises a plurality of data packets **402**. Since the particular data logged in each data packet **402** can vary, these data packets are shown as having varying sizes. In embodiments, when using time-travel debugging technologies, a data packet **402** records the inputs (e.g., register values, memory values, etc.) to one or more executable instructions that executed as part of this first thread of the application **113**. In embodiments, memory values are obtained as influxes to cache(s) **107** and/or as uncached reads. In embodiments, data stream **401***a* also includes one or more key frames **403** (e.g., key frames **403***a* and **403***b*) that each records sufficient information, such as a snapshot of register and/or memory values, that enables the prior execution of the thread to be replayed, starting at the point of the key frame and proceeding forward.

(48) In embodiments, an execution trace also includes the actual code that was executed as part of an application **113**. Thus, in FIG. **4**, each data packet **402** is shown as including a data inputs portion **404** (non-shaded) and a code portion **405** (shaded). In embodiments, the code portion **405** of each data packet **402**, if present, includes the executable instructions that executed based on the corresponding data inputs. In other embodiments, however, an execution trace omits the actual code that was executed, instead relying on having separate access to the code of the application **113** (e.g., from durable storage **104**). In these other embodiments, each data packet specifies an address or offset to the appropriate executable instruction(s) in an application binary image. Although not shown, it is possible that the execution trace **400** includes a data stream **401** that stores one or more of the outputs of code execution. It is noted that used of different data input and code portions of a data packet is for illustrative purposes only, and that the same data could be stored in a variety of manners, such as by the use of multiple data packets.

(49) If there are multiple data streams **401**, in embodiments these data streams can include sequencing events. Each sequencing event records the occurrence of an event that is orderable across different execution contexts, such as threads. In one example, sequencing events correspond to interactions between the threads, such as accesses to memory that is shared by the threads. Thus, for instance, if a first thread that is traced into a first data stream (e.g., **401***a*) writes to a synchronization variable, a first sequencing event is recorded into that data stream (e.g., **401***a*). Later, if a second thread that is traced into a second data stream (e.g., **401***b*) reads from that synchronization variable, a second sequencing event is recorded into that data stream (e.g., **401***b*). These sequencing events are inherently ordered. For example, in some embodiments each sequencing event is associated with a monotonically incrementing value, with the monotonically incrementing values defining a total order among the sequencing events. In one example, a first sequencing event recorded into a first data stream is given a value of one, a second sequencing event recorded into a second data stream is given a value of two, etc.

(50) Some bit-accurate tracing approaches leverage extensions to a processor cache that track whether the value of a given cache line can be considered to have been captured into an execution trace **114** on behalf of at least one processing unit. In various implementations, these cache modifications extend the entries of one or more of processor's caches to include additional “logging” bits (e.g., portion **302***c*), or reserve one or more entries for logging bit use. These logging bits enable a processor's control logic **108** to identify, for each cache line, one or more processing units that consumed/logged the cache line. Use of logging bits can enable the processor's control

logic **108** to avoid re-logging cache line influxes for one execution context after a processing unit transitions to another execution context (e.g., another thread, another virtual machine, kernel mode, etc.) if that other context did not modify the cache line. Additionally, use of logging bits can enable a trace entry for one context to reference data already logged on behalf of another context.
(51) Additional, or alternative, bit-accurate tracing approaches use memory markings as logging cues. More particularly, in embodiments, the debugging component **110** and the control logic **108** cooperate to record replayable execution trace(s) **114**/**114**′ based on the categorization component **111** categorizing different memory regions, such as physical memory pages in main memory **103**, as logged or not logged. These categorizations are represented by region categorizations **115** within main memory **103**. In embodiments, an execution context corresponds to at least one of a process executing on top of an operating system **109***b*, an operating system **109***b*, a virtual machine/memory partition created by the hypervisor **109***a*, an enclave, a nested hypervisor, and the like. In embodiments, using memory markings as logging cues for processor-based execution tracing is based at least on (i) the categorization component **111** maintaining region categorizations **115** (which categorize different memory regions as being logged and not logged), and on (ii) the control logic **108** using these region categorizations **115** to make logging decisions during tracing.
(52) Additional, or alternative, bit-accurate tracing approaches utilize associative caches, coupled with processor cache way-locking features of some processors to reserve a subset of the cache for an entity that being traced, and then logs cache misses relating to that entity into a reserved subset of the cache. In particular, some bit-accurate tracing approaches utilize way-locking to reserve one or more cache "ways" for an entity that is being traced, such that the locked/reserved ways are used exclusively for storing cache misses relating to execution of that entity. Thus, by virtue of which way(s) to which a cache entry belongs, embodiments can determine whether or not a corresponding cache line has been logged.
(53) Regardless of which tracking technique(s) are used, in embodiments the control logic **108** (and/or a co-processor cooperating with control logic **108**) logs based on influxes at a particular level in a multi-level cache. For example, in embodiments the control logic **108** logs influxes at an L2 cache level, even if one or more higher cache levels are present. In general, logging influxes to a cache with relatively more cache entries results in smaller traces than logging influxes to a cache with relatively fewer cache entries. This is because a larger cache generally has fewer evictions than a smaller cache, and thus the larger cache has fewer influxes of the same cache data (and thus, there is less duplicate logging of the same cache line data). As such, from a trace size perspective, it is often desirable to log at a higher (upper) cache level (which is generally larger in size than a lower cache level). However, from an implementation and cost perspective, it is often desirable to implement logging at a lower cache level. For example, it may be less costly (e.g., in terms of processor die size) to implement tracking mechanisms at a lower cache level than it is a higher cache level.
(54) I. Trace Logging Using Tags in a Higher Memory Tier
(55) The embodiments herein strike a balance between these competing goals by implementing control logic **108** (and/or a co-processor) that intelligently determines whether or not to log an influx of a cache line into a recording first cache level based on using a tag in a higher memory tier (e.g., an upper second cache level, or main memory **103**) to determine if a value of the cache line that is being influxed has been previously captured into a trace, such as in connection with a prior influx of the cache line to the first cache level.
(56) To demonstrate some embodiments of how the control logic **108** (and/or a co-processor) accomplishes the foregoing, FIG. **1**B illustrates an example computing environment **100***b* showing additional detail of control logic **108**, including components that embodiments of the control logic **108** uses when interacting with the cache **107**. The depicted components of control logic **108**, together with any sub-components, represent various functions that the control logic **108** might implement or utilize in accordance with various embodiments described herein. It will be

appreciated, however, that the depicted components—including their identity, sub-components, and arrangement—are presented merely as an aid in describing various embodiments of the control logic **108** described herein, and that these components are non-limiting to how software and/or hardware might implement various embodiments of the control logic **108** described herein, or of the particular functionality thereof.
(57) As shown, the control logic **108** comprises cache influx logic **118** that operates to influx a cache line of data into a recording cache level (i.e., a cache level into which influxes are being logged), and cache eviction logic **124** that operates to evict a cache line of data from the recording cache level. In some embodiments, the control logic **108** supports the enabling and disabling of recording features of a processor, which may be supported globally, per-processing unit, per execution context, etc. Thus, the cache influx logic **118** is shown as comprising recording influx logic **119** that operates when infuxing a cache line to a cache that is currently recording, and as potentially comprising non-recording influx logic **123** that operates when infuxing a cache line to a cache that is not currently recording. Similarly, the cache eviction logic **124** is shown as comprising recording eviction logic **125** that operates when evicting a cache line from a cache that is recording, and as potentially comprising non-recording eviction logic **128** that operates when evicting a cache line from a cache that is not recording.
(58) Turning to the cache influx logic **118**, the recording influx logic **119** comprises a tag determination component **120**, logged value logic **121**, and non-logged value logic **122**. In general, when a cache line is being influxed from an upper-level cache (e.g., an L3 cache in FIG. **2**) into a cache level that is being logged (e.g., an L2 cache in FIG. **2**), the tag determination component **120** reads a tag in the higher memory tier (e.g., an additional portion **302***c* in an upper-level cache, or one of tags **117** in main memory **202**) to determine if there is an indicium within the tag that a value of the cache line being influxed from the upper-level cache has been previously captured by an execution trace **114**. In some embodiments, a value of the cache line is considered to have been previously captured by an execution trace **114** if the present value of the cache line in an upper-level cache is recorded in the execution trace **114**. An additional, or alternative, embodiments a value of the cache line is considered to have been previously captured by an execution trace **114** if the present value of the cache line in the upper-level cache can be reconstructed based on the execution trace **114**—such as by obtaining a prior value of the cache line from the execution trace **114**, and then replaying one or more executable instructions based on the execution trace **114** to transform that cache line to arrive at the present value of the cache line in the upper-level cache. The indicium can vary by implementation and comprises value(s) stored within one or more fields of the tag. For example, in embodiments the tag comprises at least one of a field for storing an indication of whether or not the cache line has been logged (e.g., a single bit "logged" flag), a field for storing an address space identifier (ASID) for which the cache line has been logged, a field for storing a virtual machine identifier (VMID) for which the cache line has been logged, a field for storing an exception level (e.g., ARM processors), a field for storing a ring (e.g., x86 processors), or a field for storing a security state (e.g., ARM processors).
(59) In embodiments, if the tag determination component **120** identifies, within the tag, one or more indicia that the cache line was logged (e.g., a logged flag being set, the presence of an ASID, the presence of a VMID, etc.), then the tag determination component **120** further determines if the cache line has definitely not been modified after a most recent prior eviction from any recording cache level (e.g., based on the tag having been modified, based on CCP state stored in the tag or elsewhere, etc.). When the tag determination component **120** identifies an indicium that the cache line was logged, and when the tag determination component **120** further determines that the cache line has definitely not been modified after a most recent prior eviction from any recording cache level, then the tag determination component **120** concludes that the value of the cache line has been previously captured by an execution trace **114**. In this case, the cache influx logic **118** follows a logic path defined by the logged value logic **121**. In general, the logged value logic **121** handles an

influx of the cache line while refraining from logging a value of the cache line into an execution trace **114**. Even though the logged value logic **121** does not log the value of the cache line, in embodiments the logged value logic **121** does take appropriate action to indicate that the cache line has been logged, such as by appropriately setting tracking bits associated with an entry into which the cache line was stored, by influxing the cache line into a logged way, etc. In embodiments, the logged value logic **121** may store, into an execution trace **114**, a reference to prior-logged value of the cache line.
(60) On the other hand, if the tag determination component **120** cannot identify an indicium that the cache line was logged, or if the tag determination component **120** cannot definitively determine that the cache line has not been modified after a prior eviction from the recording cache level (e.g., the cache line was definitely not logged, or it is indeterminate as to whether the cache line was logged), then the tag determination component **120** concludes that the value of the cache line has not been previously captured by an execution trace **114**. In this case, the cache influx logic **118** follows a logic path defined by the non-logged value logic **122**. In general, the non-logged value logic **122** handles an influx of the cache line as appropriate for a cache line that has not been previously logged. In embodiments, the non-logged value logic **122** operates in substantially the same manner as prior bit-accurate tracing approaches that lacked a consideration of tags in a higher memory tier. Thus, the particular action (or inaction) of the non-logged value logic **122** can vary depending on the tracing approach being used, such as logging bits, memory page marking, way locking, etc. In some embodiments, the non-logged value logic **122** captures a value of the cache line into an execution trace **114** in connection with performing the influx and takes appropriate action to indicate that the cache line has been logged (e.g., by appropriately setting tracking bits associated with an entry into which the cache line was stored, by influxing the cache line into a logged way, etc.). In other embodiments, there is separate control logic **108** that will capture the value of the cache line based on a subsequent trigger, so the non-logged value logic **122** influxes the cache line without capturing a value of the cache line into an execution trace **114** and/or without indicating that the cache line is logged. At times, the non-logged value logic **122** may refrain from logging the cache line altogether.
(61) Turning to the cache eviction logic **124**, the recording eviction logic **125** comprises a logged determination component **126** and a cache line tagging component **127**. In general, when a cache line is being evicted from a cache level that is being logged (e.g., an L2 cache in FIG. **2**) to an upper-level cache (e.g., an L3 cache in FIG. **2**), the logged determination component **126** determines if a current value of that cache line has been captured into an execution trace **114** at the recording cache level. In some embodiments, the logged determination component **126** determines that a current value of the cache line has been captured by the execution trace **114** if the current value of the cache line in the recording cache level is recorded in the execution trace **114** (e.g., in cases where the cache line was logged during an influx, and the cache line was not modified prior to the eviction). In additional, or alternative, embodiments the logged determination component **126** determines that a current value of the cache line has been captured by the execution trace **114** if the current value of the cache line in the recording cache level can be reconstructed based on the execution trace **114** (e.g., in cases where the cache line was logged during an influx, but the cache line was modified by logged instructions prior to the eviction).
(62) In embodiments, if the current value of that cache line has been captured, then the cache eviction logic **124** may choose to set a tag in a higher memory tier (e.g., an additional portion **302***c* in an upper-level cache, or one of tags **117** in main memory **202**) with an indicium that a value of the cache line in the upper-level cache to which the cache line was evicted has been previously captured by an execution trace **114**. Otherwise, the cache eviction logic **124** ensures that the tag in the higher memory tier indicates that the value of the cache line in the upper-level cache has not been logged. Thus, depending on the determination by the logged determination component **126**, the cache line tagging component **127** may set one or more fields within a tag to indicate whether

or not the value of the cache line in the upper-level cache has been logged. In embodiments, this includes setting at least one of a field for storing an indication of whether or not the cache line has been logged (e.g., a single-bit "logged" flag), a field for storing an ASID for which the cache line has been logged, a field for storing a VMID for which the cache line has been logged, a field for storing an exception level (e.g., ARM processors), a field for storing a ring (e.g., x86 processors), or a field for storing a security state (e.g., ARM processors). In embodiments, a lack of an ASID or a VMID in the second and/or third fields indicates that the cache line has not been logged. In embodiments, when tag data is stored in main memory, the data stored is sufficient to determine whether a cache line was captured or not (e.g., a single bit flag). In embodiments, tag data in main memory (e.g., tags **117**) is stored in one or more data structures, such as one or more bitmaps, one or more tree structures (e.g., similar to page table structures), and the like.

(63) In some alternative embodiments, the cache influx logic **118**, rather than the cache eviction logic **124** handles updating of tags to indicate when the values of cache lines in an upper-level cache have been logged. For example, in some embodiments, in connection with logging a cache line, the non-logged value logic **122** operates much like the cache line tagging component **127** to set one or more fields within a tag in a higher memory tier to indicate that the cache line has been logged. In some embodiments, there is separate logging and influx logic, such that there are independent logging and influx operations. In these embodiments, the act of logging and setting tracking information (e.g., logging bits) may also set a tag in a higher memory tier (or trigger and eventual update of the tag). In embodiments, logging actions ensure that there is consistency between a cache line's logging status and a tag in the higher memory tier, even if those logging actions are not made in connection with a cache influx. For example, if a cache line's "logged" status is cleared (e.g., due to a write by a non-logged context) while it is in a recording cache level, then a corresponding tag is also cleared (or eventually cleared) in the higher memory tier; later, if the cache line's "logged" status is set, then the corresponding tag is also set (or eventually set) in the higher memory tier.

(64) Operation of the control logic **108** is now described in greater detail in connection with FIGS. **5**A and **5**B which illustrate methods of cache-based trace logging using tags in a higher memory tier. The following discussion now refers to a number of methods and method acts. Although the method acts may be discussed in certain orders, or may be illustrated in a flow chart as occurring in a particular order, no particular ordering is required unless specifically stated, or required because an act is dependent on another act being completed prior to the act being performed.

(65) FIG. **5**A illustrates a flow chart of an example method **500***a* for using tags in a higher memory tier to determine whether or not to log a cache line influx. FIG. **5**B illustrates a flow chart of an example method **500***b* for setting tags in a higher memory tier during a cache line eviction. In some embodiments, method **500***a* and method **500***b* are treated as entirely separate methods. In other embodiments, method **500***a* and method **500***b* are treated as being part of a single combined method. In embodiments, logic for implementing method **500***a* and/or **500***b* are encoded as control logic **108** that configures a processor (e.g., processor **102** comprising processing unit **106** and cache **107**, or a co-processor) to perform the method.

(66) Methods **500***a* and **500***b* refer to a first cache level that is a recording cache level, and a higher memory tier. In various embodiments, the higher memory tier is a second cache level arranged as an upper cache level to the first cache level, or main memory (e.g., main memory **103**). In one example, and referring to FIG. **2**, the first cache level is an L1 cache and the higher memory tier is an L2 cache, an L3 cache, or higher (e.g., an L4 cache or main memory **202**). In another example, the first cache level is an L2 cache and the higher memory tier is an L3 cache, or higher. In another example, the first cache level is an L3 cache and the higher memory tier is an L4 cache, or higher. Methods **500***a*/**500***b* will now be described with respect to the components and data of computing environments **100***a* and **100***b* and the example computing environment **200** of FIG. **2**.

(67) Turning initially to FIG. **5**A, method **500***a* begins at an act **501** where there is a cache line to

be influxed to a first cache level. In general, act **501** comprises influxing a cache line into a first cache level. In some embodiments, act **501** comprises influxing a cache line into a first cache level from a second cache level arranged as an upper cache level to the first cache level. For example, based on activity by processing unit A1, a cache line is influxed from cache L3-A to cache L2-A1; in this example, the first cache level is an L2 cache, and the higher memory tier is an L3 cache.
(68) As mentioned, some embodiments enable processor recording features to be enabled or disabled, such as globally, per-processing unit, per execution context, etc. In these embodiments, method **500*a*** proceeds to an act **502** of determining if a recording feature is enabled. In an example, the cache influx logic **118** determines if trace recording is enabled or disabled, such as by checking a register value or some other toggleable value. When method **500*a*** comprises act **502**, it will be appreciated that the first cache level referred to in act **501** is a recording cache level only when a recording feature of the processor is enabled.
(69) If method **500*a*** comprises act **502**, and if the recording feature is determined to not be enabled in act **502**, then in embodiments method **500*a*** proceeds to an act **503** of influxing with non-recording logic (i.e., using non-recording influx logic **123**), which in embodiments ignores any tags associated with the influxed cache line (at least for recording purposes). Conversely, if method **500*a*** comprises act **502**, and if the recording feature is determined to be enabled in act **502**, or if method **500*a*** lacks act **502** (i.e., a recording feature is always active), then in embodiments method **500*a*** proceeds to an act **504** of influxing with recording logic (i.e., using recording influx logic **119**).
(70) As shown, act **504** comprises an act **505** of reading a tag in a higher memory tier. In general, act **505** comprises, based at least on the first cache level being a recording cache level, reading a tag that is stored in a higher memory tier and that is associated with the cache line. In some embodiments, act **502** comprises reading a tag that is stored in the second cache level and that is associated with the cache line in the second cache level. For example, the tag determination component **120** reads a tag within cache L3-A, and which is associated with the cache line in cache L3-A that is being influxed into cache L2-A1. In other embodiments, act **502** comprises reading a tag that is stored in main memory and that is associated with the cache line in the second cache level. In an example, the tag determination component **120** reads a tag within main memory **202** (e.g., tags **117**), and which is associated with the cache line in cache L3-A that is being influxed into cache L2-A1.
(71) Act **504** proceeds to an act **506** of determining if the cache line is indicated in the higher memory tier as being logged. In general act **506** comprises, based at least on reading the tag stored in the higher memory tier, determining whether a first value of the cache line has been previously captured by a trace. In some embodiments, act **506** comprises, based at least on reading the tag, determining whether a first value of the cache line within the second cache level has been previously captured by a trace. For example, the tag determination component **120** determines, from the tag read in act **505**, if there is an indicium within the tag that a value of the cache line, as influxed from cache L3-A into cache L2-A1 in act **501**, has been previously captured to an execution trace **114** in connection with a prior influx to cache L2-A1. If so, the tag determination component **120** also determines if the cache line has definitely not been modified in an upper cache level (e.g., cache L3-A) after a prior eviction from cache L2-A1. If there is an indicium that a value of the cache line has been previously captured, and if the cache line has definitely not been modified in an upper cache level after a prior eviction from cache L2-A1, then the tag determination component **120** concludes that the cache line is indicated in the higher memory tier as being logged (i.e., “Yes” from act **506**). Otherwise, the tag determination component **120** concludes that the cache line is not indicated in the higher memory tier as being logged (i.e., “No” from act **506**). In embodiments, the first value of the cache line is determined to have been previously captured by the trace when CCP data indicates that the cache line has not been modified within an upper second cache level, and the first value of the cache line is determined to have not

been previously captured by the trace when the CCP data indicates that the cache line could have been modified within the upper second cache level.

(72) Depending on the determination of act **506**, act **504** either comprises an act **507** of influxing with the cache line value not certainly known to have been already captured by a trace (i.e., following the "No" path from act **506**), or an act **508** of influxing with the cache line value known to have been already captured by a trace (i.e., following the "Yes" path from act **506**).

(73) In some embodiments act **507** comprises, when the first value of the cache line is determined to have not been previously captured by the trace, following a non-logged value logic path when influxing the cache line into the first cache level. In an example, cache influx logic **118** follows a logic path defined by the non-logged value logic **122** when influxing the cache line, and thus cache line is influxed to cache L2-A1 while taking an appropriate logging action (if any). Accordingly, in act **507**, the non-logged value logic path stores the cache line into an entry within the first cache level while initiating logging of the first value of the cache line into the trace. In some embodiments, the non-logged value logic **122** updates the tag in the higher memory tier to indicate that the cache line has been logged. Thus, in some embodiments, the non-logged value logic path ensures that the tag stored in the higher memory tier indicates that the cache line has been logged. In an example, the non-logged value logic **122** sets one or more fields within a tag in cache L3-A, or within a tag (e.g., tags **117**) in main memory **202**, to indicate that a value of the cache line within cache L3-A has not been logged, such as by appropriately setting or clearing a "logged" flag, ensuring that an ASID field is clear or has changed, or ensuring that a VMID field is clear or has changed.

(74) In some embodiments act **508** comprises, when the first value of the cache line is determined to have been previously captured by the trace, following a logged value logic path when influxing the cache line into the first cache level. In an example, cache influx logic **118** follows a logic path defined by the logged value logic **121** when influxing the cache line, and thus cache line is influxed to cache L2-A1 while refraining from logging a value of the cache line into an execution trace **114**. Accordingly, in act **507**, the logged value logic path stores the cache line into an entry within the first cache level without initiating logging of the first value of the cache line into the trace. Notably, it is possible that the logged value logic **121** stores some record of the influx, such as by storing a reference to a prior-logged value of the cache line (e.g., a prior logged influx by processing unit A2, for instance). Accordingly, in some embodiments of act **508**, the logged value logic path stores the cache line into an entry within the first cache level while initiating logging, into the trace, a reference to the first value of the cache line previously captured by the trace.

(75) Whether following the logged value logic **121** or the non-logged value logic **122**, in embodiments the cache influx logic **118** may take appropriate action to indicate that the cache line has been logged, such as by appropriately setting tracking bits associated with an entry into which the cache line was stored, by influxing the cache line into a logged way, etc. Thus, in embodiments, influxing the cache line into the first cache level also includes, based at least on the first cache level being a recording cache level, at least one of storing the cache line within a logging way of the first cache level, or setting one or more tracking bits associated with an entry in the first cache level that stores the cache line to indicate that the cache line has been logged.

(76) Regardless of whether method **500***a* influxed with non-recording logic in act **503** or influxed with recording logic in act **504**, in embodiments method **500***a* comprises act **509**, which proceeds to either act **501** (i.e., to process an influx of an additional cache line), or an act **510** of method **500***b* (i.e., to process an eviction of a cache line).

(77) Turning now to FIG. 5B, method **500***b* begins at act **510** where there is a cache line to be evicted from the first cache level. In some embodiments, act **510** comprises determining that a cache line in a first cache level is to be evicted. In an example, the control logic **108** determines that a cache line is evicted from cache L2-A1 into cache L3-A; in this example, the first cache level is an L2 cache, and the second cache level is an L3 cache.

(78) As mentioned, some embodiments enable processor recording features to be enabled or disabled, such as globally, per-processing unit, per execution context, etc. In these embodiments, method **500***b* proceeds to an act **511** of determining if a recording feature is enabled. In an example, the cache influx logic **118** determines if trace recording is enabled or disabled, such as by checking a register value or some other toggleable value. When method **500***b* comprises act **511**, it will be appreciated that the first cache level referred to in act **510** is a recording cache level only when a recording feature of the processor is enabled.
(79) If method **500***b* comprises act **511**, and if the recording feature is determined to not be enabled in act **511**, then method **500***b* proceeds to an act **512** of evicting with non-recording logic (i.e., using non-recording eviction logic **128**). Conversely, if method **500***b* comprises act **511**, and if the recording feature is determined to be enabled in act **511**, or if method **500***b* lacks act **511** (i.e., a recording feature is always active), then method **500***b* proceeds to an act **513** of evicting with recording logic (i.e., using recording eviction logic **125**).
(80) As shown, act **513** comprises an act **514** of determining if the cache line is logged in the first cache level. In some embodiments act **514** comprises, based at least on the first cache level being a recording cache level, determining whether a second value of the cache line within the first cache level has been captured by the trace. In an example, the logged determination component **126** determines if a current value of the cache line being evicted from cache L2-A1 to cache L3-A has been captured by an execution trace **114**, such as by checking logging status (e.g., logging bits, cache ways, etc.) to determine if the cache line was logged at influx, and by checking CCP data to determine if the cache line was modified while in cache L2-A1. If the cache line has been logged and its value has not changed, then the logged determination component **126** concludes that the current value of the cache line has been logged (i.e., “Yes” from act **514**). Conversely, if the cache line has not been logged or its value has changed, then the logged determination component **126** concludes that the current value of the cache line has not been logged (i.e., “No” from act **514**).
(81) Depending on the determination of act **514**, act **513** either proceeds to an act **515** of ensuring that a tag in the higher memory tier indicates the cache line as not logged (i.e., following the “No” path from act **514**), or an act **516** of ensuring that the tag in the higher memory tier indicates that cache line as logged (i.e., following the “Yes” path from act **514**).
(82) In some embodiments act **515** comprises ensuring that the tag stored in the second cache level indicates that the cache line has not been logged. In an example, the cache line tagging component **127** sets one or more fields within a tag in cache L3-A to indicate that the cache line stored in cache L3-A has not been logged, such as by appropriately setting or clearing a “logged” flag, ensuring that an ASID field is clear or has changed, or ensuring that a VMID field is clear or has changed. In other embodiments act **515** comprises ensuring that the tag stored in the main memory indicates that the cache line has not been logged. In an example, the cache line tagging component **127** sets one or more fields within a tag (e.g., tag **117**) in main memory **202** to indicate that the cache line stored in cache L3-A has not been logged, such as by appropriately setting or clearing a “logged” flag, ensuring that an ASID field is clear or has changed, or ensuring that a VMID field is clear or has changed.
(83) In some embodiments act **516** comprises, based at least on the second value of the cache line having been captured by the trace, ensuring that the tag stored in the higher memory tier indicates that the cache line has been logged. In an example, the cache line tagging component **127** sets one or more fields within a tag in cache L3-A, or in main memory **202**, to indicate that the value of cache line within cache L3-A has been logged, such as by appropriately setting or clearing a “logged” flag, setting an ASID field to an appropriate address space, or setting a VMID field to an appropriate virtual machine identifier. Thus, in embodiments, ensuring that the tag in the higher memory tier indicates that the cache line has been logged comprises at least one of setting a first field in the tag to indicate that the cache line has been logged, setting a second field in the tag to an address space identifier associated with the cache line, or setting a third field in the tag to virtual

machine identifier associated with the cache line.
(84) Notably, act **516** is shown in broken lines, indicating that, for a given cache line, the cache eviction logic **124** could choose not to set the tag in the higher memory tier to indicate that the cache line is logged—even when act **515** reaches a "Yes" determination. In these cases, the cache eviction logic **124** instead sets the tag indicate that the cache line has not been logged (i.e., act **515**). It will be appreciated that, even though a cache line could be marked as logged, doing so is not necessary for correct logging (even though this could lead to increased trace size).
(85) Regardless of whether method **500***b* evicted with non-recording logic in act **512**, or evicted with recording logic in act **513**, in embodiments method **500***b* comprises act **517**, which proceeds to either act **510** (i.e., to process an eviction of another cache line), or act **501** of method **500***a* (i.e., to process an influx of a cache line).
(86) As will be appreciated by one of ordinary skill in that art, methods **500***a*/**500***b* interact for proper handling of a given cache line. For instance, if a particular cache line is being freshly imported from main memory **202**, then when influxing the cache line to the first cache level method **500***a* would not find any indication in the higher memory tier that the cache line has been logged (act **506**), and thus method **500***a* would influx the cache line using the non-logged logic (act **507**) and log the cache line. However, if that cache line is later evicted from the first cache level to the second cache level, and its value has been captured by an execution trace **114**, then method **500***b* could ensure that a tag in the higher memory tier indicates the cache line as logged (act **516**). Then, the next time the cache line is influxed from the second cache level to the first cache level without having been modified in the second cache level, method **500***a* can influx the cache line using the logged logic (act **508**) which avoids re-logging the cache line.
(87) Thus, some embodiments comprise following the non-logged value logic path when influxing the cache line into the first cache level, and subsequently ensuring that the tag stored in the higher memory tier indicates that the cache line has been logged when evicting the cache line from the first cache level (if its value has been captured by an execution trace **114**). Additionally, some embodiments comprise ensuring that the tag stored in the higher memory tier indicates that the cache line has been logged when evicting the cache line from the first cache level, and subsequently following the logged value logic path when influxing the cache line into the first cache level.
(88) Accordingly, at least some embodiments described herein perform cache-based trace logging using tags in a higher memory tier. These embodiments operate to log influxes to a first cache level that is logging, but leverage tags within a higher memory tier to track whether a value of a given cache line influx has been previously captured. In particular, during an influx of a cache line from an upper second cache level to the first cache level, embodiments consult a tag in a higher memory tier (e.g., the second cache level or main memory) to determine if a value of the cache line within the second cache level was previously captured. If so, embodiments refrain from re-logging the cache line in connection with the influx to the first cache level. Additionally, during evictions from the first cache level to the second cache level, embodiments determine whether a value the cache line being evicted to the second cache level has been previously captured, and sets a tag in the higher memory tier as appropriate. Thus, the embodiments herein can leverage a potentially larger upper-level cache, or even main memory, to decrease trace size, while limiting implementation details and complication to a generally smaller lower cache level.
(89) II. Treating Main Memory as a Collection of Tagged Cache Lines while Trace Logging Using Tags in a Higher Memory Tier
(90) The foregoing techniques use tags in an upper memory tier (e.g., an upper cache level or main memory **103**) to track whether a value being influxed from an upper cache level (e.g., cache L3-A) to a lower cache level (e.g., L2-A1) has been previously captured by an execution trace **114**. Thus, whether using tags stored in the cache **107** or in the main memory **103**, these techniques use tags to track whether the value of a cache line in the cache **107** has been previously captured by the

execution trace **114**.

(91) In embodiments, these tagging techniques are extended to treat main memory as a collection of tagged cache lines, in order to track whether the value of a memory block within main memory **103** has been captured by an execution trace **114**, or can be otherwise reconstructed (e.g., because the memory block is initialized memory, because the memory block is backed by a file, etc.). These embodiments can be used to avoid logging some influxes from main memory **103** to a recording cache level within cache **107**, even if those influxes pass through one or more intermediary cache levels, which further reduces a size of execution traces **114**. While using tags (whether stored in cache **107** or in main memory **103**) to indicate whether a value of a corresponding cache line in the cache **107** is already captured by an execution trace have been shown to reduce trace file size by 3-4× (when compared to prior tracing techniques that lack use of tags in a higher memory tier), treating main memory as a collection of tagged cache lines for trace logging has been shown to further reduce trace file size by an additional 10-100×.

(92) In general, embodiments that treat main memory as a collection of tagged cache lines for trace logging involve the memory tagging component **112** allocating a plurality of memory blocks **116** within main memory **103**, along with a corresponding plurality of tags **117**—one or more of which indicates whether data stored in its associated memory block has been captured by an execution trace (or is otherwise recoverable). The memory tagging component **112** then synchronizes appropriate tags within tags **117** with corresponding tags in cache **107** (such that the foregoing tagging techniques remain operable) and manages those tags **117** in light of memory operations affecting the memory blocks **116**—such as DMA operations, memory paging operations, memory initialization operations, and file-mapping operations.

(93) To further demonstrate these concepts, FIG. **1**C illustrates an example computing environment **100***c* showing additional detail of a memory tagging component **112** that is configured to treat main memory as a collection of tagged cache lines while trace logging using tags in a higher memory tier. The depicted components of the memory tagging component **112** represent various functions that the memory tagging component **112** might implement or utilize in accordance with various embodiments described herein. It will be appreciated, however, that the depicted components—including their identity and arrangement—are presented merely as an aid in describing various embodiments of memory tagging component **112** described herein, and that these components are non-limiting to how software and/or hardware might implement various embodiments of memory tagging component **112** described herein, or of the particular functionality thereof.

(94) The memory tagging component **112** is now described in connection with FIG. **6**, which illustrates a flowchart of a method **600** for treating main memory as a collection of tagged cache lines for trace logging. Thus, the following discussion now refers to a method and method acts. Although the method acts may be discussed in certain orders, or may be illustrated in a flow chart as occurring in a particular order, no particular ordering is required unless specifically stated, or required because an act is dependent on another act being completed prior to the act being performed. In embodiments, instructions for implementing method **600** are encoded as executable instructions (e.g., memory tagging component **112** and/or control logic **108**) stored on one or more hardware storage devices (e.g., durable storage **104** and/or control logic **108**) that are executable by a processor (e.g., processor **102**) to cause a computer system (e.g., computer system **101**) to perform method **600**.

(95) The memory tagging component **112** is illustrated as including a memory block allocation component **130**. In embodiments, the memory block allocation component **130** allocates a plurality of memory blocks (memory blocks **116**) that functionally operate as cache lines stored within main memory **103**. Thus, in embodiments, the memory block allocation component **130** allocates memory blocks that are sufficient in size to store copies of cache lines in cache **107**.

(96) Turning to FIG. **6**, method **600** comprises an act **601** of allocating, in main memory, a collection of cache line sized memory blocks. In some embodiments, act **601** comprises allocating

a plurality of memory blocks within the main memory, including allocating an amount of memory for each of the plurality of memory blocks that is a size of each of the plurality of cache lines.

(97) The memory tagging component **112** is also illustrated as including a tag allocation component **131**. In embodiments, the tag allocation component **131** allocates a set of tags **117**, one or more of which corresponds to one of the memory blocks in memory blocks **116**. One or more of these tags **117** are used to track whether the data stored in its corresponding memory block has been captured by an execution trace **114** (or is otherwise recoverable).

(98) Turning to FIG. **6**, method **600** comprises an act **602** of allocating, in main memory, a plurality of tags associated with the collection memory blocks. In some embodiments, act **602** comprises allocating a plurality of tags within the main memory, each tag being associated with one of the plurality of memory blocks, each tag indicating whether data stored in its associated memory block has been captured by an execution trace.

(99) In some embodiments, each of the tags **117** is structured similar to the tags discussed above as being stored in a cache **107**, such as a tag including one or more data fields, which include (as examples) at least one of a field for storing an indication of whether or not the memory block has been captured (e.g., a single bit "logged" flag), a field for storing an ASID for which the memory block has been logged, a field for storing a VMID for which the memory block has been logged, etc. Thus, in some embodiments of act **602**, each of the plurality of tags comprises a plurality of fields, including one or more of a first field to indicate that the memory block has been captured, a second field to store an ASID associated with the memory block, or a third field to store a VMID associated with the memory block. In other embodiments, each of the tags **117** is a single bit which operates as a flag that is set or cleared to indicate whether or not a corresponding memory block has been captured. Thus, in some embodiments of act **602**, each of the plurality of tags is a single bit that stores either a first value (e.g., a zero) indicating that data stored in a corresponding memory block has not been captured by the execution trace or a second value (e.g., a one) indicating that data stored in the corresponding memory block has been captured by the execution trace.

(100) The memory tagging component **112** is also illustrated as including a cache tag synchronization component **132**. Correspondingly, the control logic **108** (FIG. **1**B) is illustrated as including main memory tag synchronization logic **129**. In embodiments, the cache tag synchronization component **132** and the main memory tag synchronization logic **129** cooperate to synchronize any tags stored in the cache **107** and the tags **117**.

(101) Turning to FIG. **6**, method **600** comprises an act **603** of synchronizing one or more of the tags with a memory cache. In some embodiments, act **602** comprises synchronize at least one of the plurality of tags within the main memory with at least one cache line tag within the memory cache.

(102) In embodiments, when evicting a cache line from the cache **107** to the main memory **103**, one or both of the cache tag synchronization component **132** or the main memory tag synchronization logic **129** operate to update a tag in tags **117** to reflect a logged status of that cache line as was stored in a tag in the cache **107**. Thus, in some embodiments of act **603**, synchronizing the at least one tag within the main memory with the at least one cache line tag within the memory cache comprises the processor initiating an eviction of data stored in a particular cache line of the plurality of cache lines to the particular memory block. Then, based at least on initiating the eviction of the data stored in the particular cache line to the particular memory block, the processor sets the particular tag to indicate whether data stored in the particular memory block has been captured by the execution trace. Here, the processor sets the particular tag based on a value of a cache line tag in the memory cache.

(103) Similarly, in embodiments, when influxing a cache line from the main memory **103** to the cache **107**, one or both of the cache tag synchronization component **132** or the main memory tag synchronization logic **129** operate to update a tag in the cache **107** to reflect a logged status of the data being stored into the cache, based on the value of a corresponding tag in tags **117**. Thus, in

some embodiments of act **603**, synchronizing the at least one tag within the main memory with the at least one cache line tag within the memory cache comprises the processor initiating an influx of data stored in the particular memory block into a particular cache line of the plurality of cache lines. Then, based at least on initiating the influx of the data stored in the particular memory block into the particular cache line, the processor sets a cache line tag in the memory cache to indicate whether data stored in the particular cache line has been captured by the execution trace. Here, the processor sets the cache line tag based on a value of the particular tag.
(104) In some embodiments, as part of act **603**, the main memory tag synchronization logic **129** directly accesses (e.g., reads or updates) the tags **117** stored in main memory **103**. In these embodiments, the main memory tag synchronization logic **129** is aware of a location of the tags **117** because the cache tag synchronization component **132** has previously provided a base address to the tags **117** in main memory **103** to the memory tag synchronization logic **129** by writing that base address to one or more of registers **106***a*. Thus, in some embodiments, the cache tag synchronization component **132** provides the main memory tag synchronization logic **129** the base address of the tags **117**, such that method **600** comprises storing, into one or more processor registers a location in the main memory of the plurality of tags. In some embodiments, the cache tag synchronization component **132** may similarly provide the memory tag synchronization logic **129** with a location of the memory blocks **116**, such that method **600** comprises storing, into one or more processor registers a location in the main memory of the plurality of memory blocks.
(105) In other embodiments act **603** operates based on communications between the main memory tag synchronization logic **129** and the cache tag synchronization component **132**. For example, during an eviction, the main memory tag synchronization logic **129** provides the cache tag synchronization component **132** with an appropriate value to write to a tag **117** in main memory **103**, and during an influx the cache tag synchronization component **132** provides the main memory tag synchronization logic **129** with an appropriate value to write to a tag in the cache **107**.
(106) Notably, by keeping tags stored in the cache **107** (e.g., an upper second cache level) in sync with tags in main memory **103**, act **603** enables full interoperability between method **600** and one or more of methods **500***a* or method **500***b*.
(107) For example, in embodiments, when cache L2-A1 causes an influx from main memory **103** (and, as an intermediary, cache L3-A) that did not already exist in the cache **107**, the main memory tag synchronization logic **129** synchronizes a tag in cache L3-A with one of tags **117**. Thus, when the tag determination component **120** reads that tag from cache L3-A, it reads a value originating from tags **117**. In embodiments, during a later eviction from cache L3-A, the main memory tag synchronization logic **129** synchronizes the value of the tag from cache L3-A to tags **117** in main memory **103**. As such method **500***a* and method **500***b* can operate as described above, while leveraging memory blocks **116** and tags **117**.
(108) In some embodiments, the cache line is influxed directly to cache L2-A1 (bypassing any intermediary cache), and method **500***a* is extended to leverage the memory blocks **116** and tags **117** in this situation. For instance, referring to method **500***a*, in some embodiments act **501** comprises influxing a cache line into the first cache level from main memory. For example, based on activity by processing unit A1, a cache line is influxed from main memory **202** cache L2-A1 (e.g., where cache L3-A is not present). In these embodiments, the tag determination component **120** may read the value of one of tags **117** directly, in order to determine if the value being influxed (i.e., one of memory blocks **116**) has been previously captured. Thus, in some embodiments, act **506** comprises, based at least on reading the tag, determining whether a first value of the cache line within system memory has been previously captured by a trace. In embodiments, during a later eviction from cache L2-A1 to main memory **103**, in act **510** the control logic **108** determines that a cache line is evicted from cache L2-A1 into main memory **202** (e.g., where cache L3-A is not present). Here, the cache line tagging component **127** is extended to update tags **117** as part of acts **515** and **516**.
(109) Notably, it is possible that one or more of the memory blocks **116** could be modified while in

main memory **103** (e.g., due to a DMA operation on the memory block), or for one or more of the memory blocks **116** to be paged out (e.g., to durable storage **104**). Additionally, it is possible that a value of one or more of the memory blocks **116** is knowable or reconstructable even if it is not stored in an execution trace **114** (e.g., due to that memory block being initialized by operating system **109***b*, or due to that memory block being mapped to a file). Thus, the memory tagging component **112** is illustrated as including a plurality of handlers (i.e., DMA handler **133**, paging handler **134**, memory initialization handler **135**, and known memory handler **136**) which are configured to managing a traced status of memory blocks **116** in each of these situations.

(110) Turning to FIG. **6**, method **600** comprises an act **604** of managing a traced status of at least one memory block. In embodiments, act **604** is carried out by one or more of the DMA handler **133**, the paging handler **134**, the memory initialization handler **135**, or the known memory handler **136**.

(111) The DMA handler **133** detects when any of memory blocks **116** are modified by a DMA hardware device. When a memory block is modified by DMA, the DMA handler **133** ensures that any corresponding tag in tags **117** is set to indicate that the corresponding memory block is not captured in an execution trace **114**.

(112) Turning to FIG. **6**, in embodiments act **604** comprises an act **604***a* of managing trace status in connection with DMA operation(s) on allocated memory block(s). In embodiments, act **604***a* comprises, based at least on identifying a DMA operation modifying a particular memory block of the plurality of memory blocks, setting a particular tag of the plurality of tags that is associated with the particular memory block to a value (e.g., a zero) indicating that data stored in the particular memory block has not been captured by the execution trace.

(113) While act **604***a* is shown in broken lines to indicate that it may not occur with each instance of act **604**, in some embodiments act **604***a* does occur, such that managing the traced status of the plurality of memory blocks in act **604** comprises identifying the DMA operation modifying the particular memory block. Then, based at least on identifying the DMA operation, act **604** comprises setting the particular tag to a value (e.g., a zero) indicating that data stored in the particular memory block has not been captured by the execution trace.

(114) When a DMA operation modifies one of memory blocks **116**, it is a hardware device—such as a network device—that modifies that memory block. Thus, in some embodiments, it is that hardware device that is responsible for setting any relevant tag(s). In embodiments, the hardware device is configured to set any relevant tag(s) based on the hardware device being a CCP participant and based on the main memory tag synchronization logic **129** being made aware of the location of the tags **117** in main memory **103**. Thus, in some embodiments of act **604***a*, a hardware device that caused the DMA operation sets the particular tag to a value (e.g., a zero) indicating that data stored in the particular memory block has not been captured by the execution trace.

(115) In other embodiments, the operating environment **109**, itself, is responsible for setting any relevant tag(s), such as via the DMA handler **133**. Thus, in these other embodiments of act **604***a*, an operating system sets the particular tag to a value (e.g., a zero) indicating that data stored in the particular memory block has not been captured by the execution trace. It is noted that, if the operating environment **109** is responsible for setting any relevant tag(s), then in embodiments the DMA handler **133** is configured to handle a situation in which the operating environment **109** is made aware that a DMA operation is forthcoming, but in which the exact timing of the DMA operation is unknown. In embodiments, the DMA handler **133** does this by causing the memory tagging component **112** to cease tracking the logged status of relevant memory blocks when a DMA operation on those memory blocks is expected, and by resuming tracking those memory blocks only when the DMA operation is known to have occurred. Thus, for a time period from when the DMA operation is expected to when the DMA operation completes, trace recording proceeds without use of tags **117** for those memory blocks.

(116) The paging handler **134** detects when any of memory blocks **116** are affected by one or more

memory paging operations, such as being paged-out from main memory **103** to durable storage **104**, or being paged-in from durable storage **104** to main memory **103**. Based on the nature of memory paging operation, and on additional configuration of the paging handler **134** (discussed infra), the paging handler **134** sets any relevant tags in tags **117** accordingly.

(117) Turning to FIG. **6**, in embodiments act **604** comprises an act **604***b* of managing trace status in connection with memory paging involving allocated memory block(s). In embodiments, act **604***b* comprises, based at least on identifying a memory page-in operation affecting the particular memory block, setting the particular tag based on whether a paged-in value has been captured by the execution trace.

(118) While act **604***b* is shown in broken lines to indicate that it may not occur with each instance of act **604**, in some embodiments act **604***b* does occur, such that managing the traced status of the plurality of memory blocks in act **604** comprises identifying the memory page-in operation affecting the particular memory block.

(119) In some embodiments, the paging handler **134** does not maintain tagging state when one of memory blocks **116** is paged-out, so the paging handler **134** is configured to detect when one of memory blocks **116** is paged-in and set a corresponding tag in tags **117** to indicate that a value of the memory block is not captured by an execution trace **114**. Thus, in some embodiments of act **604***b*, based at least on identifying the memory page-in operation, the paging handler **134** sets the particular tag to a value (e.g., a zero) indicating that data stored in the particular memory block has not been captured by the execution trace based at least on it being unknown whether the paged-in value has been captured by the execution trace.

(120) In other embodiments, the paging handler **134** is configured to maintain (e.g., in main memory **103** or in durable storage **104**) tagging state when any of memory blocks **116** are paged-out. In these embodiments, the paging handler **134** is also configured to detect when any of memory blocks **116** are paged-in, and to set corresponding tag(s) in tags **117** based on this prior-saved state. Thus, in these embodiments, prior to identifying the memory page-in operation affecting the particular memory block in act **604***b*, the paging handler **134** identifies a memory page-out operation affecting the particular memory block. Based at least on identifying the memory page-out operation, the paging handler **134** saves a value of the particular tag. Then, after detecting the memory page-out operation in act **604***b*, the paging handler **134** identifies the memory page-in operation affecting the particular memory block. Based at least on identifying the memory page-in operation, the paging handler **134** sets the particular tag to the saved value. In some embodiments, the operating system **109***b* has full discretion of where tagging stated is stored. In other embodiments, hardware indicates allowable (or exclusion) areas in main memory **103** for storing the tagging state. In some embodiments, hardware may require specific associations between physical memory pages and tagging state indicators.

(121) In yet other embodiments, the paging handler **134** is configured to always record the values of memory blocks **116** into execution trace **114** when those blocks are paged-out (at least when logging features are enabled/active). In these embodiments, the paging handler **134** is also configured to detect when any of memory blocks **116** are paged-in, and to set corresponding tag(s) in tags **117** to indicate that the value(s) of these memory block(s) have been captured by execution trace **114**. Thus, in these embodiments, prior to identifying the memory page-in operation affecting the particular memory block in act **604***b*, the paging handler **134** identifies a memory page-out operation affecting the particular memory block. Based at least on identifying the memory page-out operation, the paging handler **134** initiates logging of the data stored in the particular memory block. Then, after detecting the memory page-out operation in act **604***b*, the paging handler **134** identifies the memory page-in operation affecting the particular memory block. Based at least on identifying a memory page-in operation, the paging handler **134** sets the particular tag to a value (e.g., a zero) indicating that data stored in the particular memory block has not been captured by the execution trace.

(122) In some embodiments, the operating system **109***b* initializes memory (e.g., to null bytes) when allocating that memory to a thread or process. In these cases, the contents of this memory are known (or at least reconstructable with knowledge of the memory initialization routine), even if those contents are not captured by an execution trace **114**. In embodiments, the memory initialization handler **135** identifies block(s) of memory in memory blocks **116** that have been initialized. In some embodiments, the memory initialization handler **135** then sets any corresponding tag(s) in tags **117** in indicate that the values of those memory block(s) are captured by an execution trace **114** (e.g., because they are constructible). In additional, or alternative embodiments, the memory initialization handler **135** leverages bit-accurate tracing approaches that use memory markings as logging cues, and sets one or more of region categorizations **115** to indicate that a memory page containing those memory block(s) is not logged.
(123) Turning to FIG. **6**, in embodiments act **604** comprises an act **604***c* of managing trace status in connection with initialization of allocated memory block(s). In embodiments, act **604***c* comprises, based at least on initializing the data stored in the particular memory block, perform one or more of (i) setting an indication that a memory page containing the particular memory block is not logged, or (ii) setting the particular tag to a value (e.g., a one) indicating that data stored in the particular memory block has been captured by the execution trace.
(124) While act **604***c* is shown in broken lines to indicate that it may not occur with each instance of act **604**, in some embodiments act **604***c* does occur, such that managing the traced status of the plurality of memory blocks in act **604** comprises initializing the data stored in the particular memory block. In some embodiments, based at least on initializing the data stored in the particular memory block, the memory initialization handler **135** sets the indication that the memory page containing the particular memory block is not logged. In other embodiments, based at least on initializing the data stored in the particular memory block, memory initialization handler **135** sets the particular tag to a value (e.g., a one) indicating that data stored in the particular memory block has been captured by the execution trace.
(125) In some embodiments, the operating system **109***b* maps one or more of memory blocks **116** to a file. Similar to the memory initialization situation just described, in these cases, the contents of this memory are also known (i.e., by access to the file), even if those contents are not captured by an execution trace **114**. In embodiments, the known memory handler **136** identifies block(s) of memory in memory blocks **116** that have been mapped to a file. In some embodiments, the known memory handler **136** then sets any corresponding tag(s) in tags **117** in indicate that the values of those memory block(s) are captured by an execution trace **114** (e.g., because they can be obtained from the file). In additional, or alternative embodiments, the known memory handler **136** leverages bit-accurate tracing approaches that use memory markings as logging cues, and sets one or more of region categorizations **115** to indicate that a memory page containing those memory block(s) is not logged.
(126) Turning to FIG. **6**, in embodiments act **604** comprises an act **604***d* of managing trace status in connection mapping file(s) to allocated memory block(s). In embodiments act **604***d* comprises, based at least on associating the particular memory block with a file, perform one or more of (i) setting an indication that the memory page containing the particular memory block is not logged, or (ii) setting the particular tag to a value (e.g., a one) indicating that data stored in the particular memory block has been captured by the execution trace.
(127) While act **604***d* is shown in broken lines to indicate that it may not occur with each instance of act **604**, in some embodiments act **604***d* does occur, such that managing the traced status of the plurality of memory blocks in act **604** comprises associating the particular memory block with the file; and based at least on associating the particular memory block with the file. In some embodiments, the known memory handler **136** sets the indication that the memory page containing the particular memory block is not logged. In other embodiments, the known memory handler **136** sets the particular tag to a value (e.g., a one) indicating that data stored in the particular memory

block has been captured by the execution trace.

(128) Accordingly, in addition to using tags to indicate whether a value of a corresponding cache line in the second cache level is already captured by an execution trace, at least some embodiments described herein also treat main memory as a collection of tagged cache lines for trace logging. These embodiments allocate a plurality of memory blocks within main memory, along with a corresponding plurality of tags—each of which indicates whether data stored in its associated memory block has been captured by an execution trace (or is otherwise recoverable). These embodiments also synchronize these tags with a memory cache and manage those tags in light of memory operations affecting the memory blocks—such as DMA operations, memory paging operations, memory initialization operations, and file-mapping operations. When compared to using tags to indicate whether a value of a corresponding cache line in the second cache level is already captured by an execution trace, treating main memory as a collection of tagged cache lines for trace logging increases a number of values that can be tracked as being previously captured (or not), which further reduces the number of cache influxes that are recorded into an execution trace. This further reduces a size of the execution trace, and further reduces processor utilization for carrying out the recording of cache influxes.

(129) The present invention may be embodied in other specific forms without departing from its essential characteristics. The described embodiments are to be considered in all respects only as illustrative and not restrictive. The scope of the invention is, therefore, indicated by the appended claims rather than by the foregoing description. All changes which come within the meaning and range of equivalency of the claims are to be embraced within their scope. When introducing elements in the appended claims, the articles “a,” “an,” “the,” and “said” are intended to mean there are one or more of the elements. The terms “comprising,” “including,” and “having” are intended to be inclusive and mean that there may be additional elements other than the listed elements.

Claims

1. A computer system for treating main memory as a collection of tagged cache lines for trace logging, comprising: a processor; a main memory; a memory cache comprising a plurality of cache lines; and one or more storage devices having stored thereon executable instructions that, when executed, cause the computer system to at least: allocate a plurality of memory blocks within the main memory, including allocating an amount of memory for each of the plurality of memory blocks that is a size of each of the plurality of cache lines; allocate a plurality of tags within the main memory, each tag being associated with one of the plurality of memory blocks, each tag indicating whether data stored in its associated memory block has been captured by an execution trace; synchronize at least one tag of the plurality of tags within the main memory with at least one cache line tag within the memory cache, including at least one of, setting a first cache line tag associated with a first cache line of the plurality of cache lines to a first value, including, based at least on the processor initiating an influx of first data stored in a first memory block of the plurality of memory blocks into the first cache line, i) reading the first value from the main memory, the first value being read from a first memory location within the main memory corresponding to a first tag of the plurality of tags that is associated with the first memory block, and ii) setting the first cache line tag within the memory cache to the first value, or setting a second tag of the plurality of tags within the main memory to a second value, including, based at least on the processor initiating an eviction of second data stored in a second cache line of the plurality of cache lines, i) reading the second value from a second cache line tag within the memory cache that is associated with the second cache line, and ii) writing the second value to the main memory, the second value being writing to a second memory location within the main memory corresponding to the second tag of the plurality of tags, wherein the second tag is associated with a second memory block of the plurality of memory blocks; and manage a traced status of at least one memory block of the

plurality of memory blocks within the main memory.
2. The computer system of claim 1, wherein each of the plurality of tags is a single bit that stores either a first bit value indicating that data stored in a particular memory block has not been captured by the execution trace, or a second bit value indicating that data stored in the particular memory block has been captured by the execution trace.
3. The computer system of claim 1, wherein managing the traced status of the plurality of memory blocks comprises, based at least on identifying a direct memory access (DMA) operation modifying a particular memory block of the plurality of memory blocks, setting a particular tag of the plurality of tags that is associated with the particular memory block to a value indicating that data stored in the particular memory block has not been captured by the execution trace.
4. The computer system of claim 3, wherein a hardware device that caused the DMA operation sets the particular tag.
5. The computer system of claim 3, wherein an operating system sets the particular tag.
6. The computer system of claim 1, wherein managing the traced status of the plurality of memory blocks comprises, based at least on identifying a memory page-in operation affecting a particular memory block and on an execution trace capture status of a paged-in value being unknown, setting a particular tag of the plurality of tags that is associated with the particular memory block to a value indicating that data stored in the particular memory block has not been captured by the execution trace.
7. The computer system of claim 1, wherein managing the traced status of the plurality of memory blocks comprises, identifying a memory page-out operation affecting a particular memory block; based at least on identifying the memory page-out operation, saving a value of a particular tag of the plurality of tags that is associated with the particular memory block; after the memory page-out operation, identifying a memory page-in operation affecting the particular memory block; and based at least on identifying the memory page-in operation, setting the particular tag to the saved value.
8. The computer system of claim 1, wherein managing the traced status of the plurality of memory blocks comprises, identifying a memory page-out operation affecting a particular memory block; based at least on identifying the memory page-out operation, initiating logging of the data stored in the particular memory block; after the memory page-out operation, identifying a memory page-in operation affecting the particular memory block; and based at least on identifying the memory page-in operation, setting a particular tag of the plurality of tags that is associated with the particular memory block to a value indicating that data stored in the particular memory block has not been captured by the execution trace.
9. The computer system of claim 1, wherein managing the traced status of the plurality of memory blocks comprises, based at least on initializing the data stored in a particular memory block, setting an indication that a memory page containing the particular memory block is not logged.
10. The computer system of claim 1, wherein managing the traced status of the plurality of memory blocks comprises, based at least on initializing the data stored in a particular memory block, setting a particular tag of the plurality of tags that is associated with the particular memory block to a value indicating that data stored in the particular memory block has been captured by the execution trace.
11. The computer system of claim 1, wherein managing the traced status of the plurality of memory blocks comprises, based at least on associating a particular memory block with a file, setting an indication that a memory page containing the particular memory block is not logged.
12. The computer system of claim 1, wherein managing the traced status of the plurality of memory blocks comprises, based at least on associating a particular memory block with a file, performing one or more of (i) setting an indication that a memory page containing the particular memory block is not logged, or (ii) setting a particular tag of the plurality of tags that is associated with the particular memory block to a value indicating that data stored in the particular memory block has been captured by the execution trace.

13. The computer system of claim 1, the one or more storage devices also having stored thereon executable instructions that, when executed, cause the computer system to store, into one or more processor registers, one or more of, a first location in the main memory of the plurality of memory blocks; or a second location in the main memory of the plurality of tags.
14. A method, implemented at a computer system that includes a processor, a main memory, and a memory cache comprising a plurality of cache lines, for treating main memory as a collection of tagged cache lines for trace logging, the method comprising: allocating a plurality of memory blocks within the main memory, including allocating an amount of memory for each of the plurality of memory blocks that is a size of each of the plurality of cache lines; allocating a plurality of tags within the main memory, each tag being associated with one of the plurality of memory blocks, each tag indicating whether data stored in its associated memory block has been captured by an execution trace; synchronizing at least one tag of the plurality of tags within the main memory with at least one cache line tag within the memory cache, including at least one of, setting a first cache line tag associated with a first cache line of the plurality of cache lines to a first value, including, based at least on the processor initiating an influx of first data stored in a first memory block of the plurality of memory blocks into the first cache line, i) reading the first value from the main memory, the first value being read from a first memory location within the main memory corresponding to a first tag of the plurality of tags that is associated with the first memory block, and ii) setting the first cache line tag within the memory cache to the first value, or setting a second tag of the plurality of tags within the main memory to a second value, including, based at least on the processor initiating an eviction of second data stored in a second cache line of the plurality of cache lines, i) reading the second value from a second cache line tag within the memory cache that is associated with the second cache line, and ii) writing the second value to the main memory, the second value being writing to a second memory location within the main memory corresponding to the second tag of the plurality of tags, wherein the second tag is associated with a second memory block of the plurality of memory blocks; and managing a traced status of at least one memory block of the plurality of memory blocks within the main memory.
15. The method of claim 14, wherein managing the traced status of the plurality of memory blocks comprises, based at least on identifying a direct memory access (DMA) operation modifying a particular memory block of the plurality of memory blocks, setting a particular tag of the plurality of tags that is associated with the particular memory block to a value indicating that data stored in the particular memory block has not been captured by the execution trace.
16. The method of claim 14, wherein managing the traced status of the plurality of memory blocks comprises, based at least on identifying a memory page-in operation affecting a particular memory block, setting a particular tag of the plurality of tags that is associated with the particular memory block based on whether a paged-in value has been captured by the execution trace.
17. The method of claim 14, wherein managing the traced status of the plurality of memory blocks comprises, based at least on initializing the data stored in a particular memory block, performing one or more of (i) setting an indication that a memory page containing the particular memory block is not logged, or (ii) setting a particular tag of the plurality of tags that is associated with the particular memory block to a value indicating that data stored in the particular memory block has been captured by the execution trace.
18. The method of claim 14, wherein managing the traced status of the plurality of memory blocks comprises, based at least on associating a particular memory block with a file, performing one or more of (i) setting an indication that a memory page containing the particular memory block is not logged, or (ii) setting a particular tag of the plurality of tags that is associated with the particular memory block to a value indicating that data stored in the particular memory block has been captured by the execution trace.
19. The method of claim 14, wherein the method comprises the setting the first cache line tag associated with the first cache line of the plurality of cache lines to the first value.

20. The method of claim 14, wherein the method comprises the setting the second tag of the plurality of tags within the main memory to the second value.
21. The computer system of claim 1, wherein the computer system sets the first cache line tag associated with the first cache line of the plurality of cache lines to the first value.
22. The computer system of claim 1, wherein the computer system sets the second tag of the plurality of tags within the main memory to the second value.